УДК 004.054
ББК 32.973.4
Ж77

Жматов Д.В. Технологии разработки программных приложений, основанные на использовании Git, Docker, Gradle [Электронный ресурс]: Практикум / Жматов Д.В., Петрова А.А. — М.: МИРЭА — Российский технологический университет, 2025.— 1 электрон. опт. диск (CD-ROM)

Данный практикум предназначен для формирования практических навыков работы с современными инструментами Git, Docker, Gradle для разработки программного обеспечения. Оно помогает студентам освоить ключевые технологии, применяемые в реальных проектах.

Практикум по дисциплине «Технология разработки программных приложений» является поддержкой ступеней квалификационного роста в бакалавриате по направлению 09.03.04 «Программная инженерия».

Практикум издается в авторской редакции.

Авторский коллектив: Жматов Дмитрий Владимирович, Петрова Анна Александровна.

Рецензенты:

Макеева Оксана Валерьевна, кандидат технических наук, проректор по качеству образования и дистанционным технологиям Автономная некоммерческая организация высшего образования «Московский региональный социально-экономический институт»

Сартаков Михаил Валериевич, кандидат технических наук, доцент кафедры Информатики и компьютерного проектирования, Федеральное государственное бюджетное образовательное учреждение высшего образования «Российский химико-технологический университет имени Д.И. Менделеева».

Системные требования:
Наличие операционной системы Windows, поддерживаемой производителем.
Наличие свободного места в оперативной памяти не менее 128 Мб.
Наличие свободного места в памяти постоянного хранения (на жестком диске) не менее 30 Мб.
Наличие интерфейса ввода информации.
Дополнительные программные средства: программа для чтения pdf-файлов (Adobe Reader).
Подписано к использованию по решению Редакционно-издательского совета
МИРЭА — Российский технологический университет.
Обьем: 3.12 мб
Тираж: 10

ISBN 978-5-7339-2592-9

© Жматов Д.В., Петрова А.А., 2025
© МИРЭА – Российский технологический университет, 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 6
1. ОСНОВЫ РАБОТЫ С GIT 7
1.1. Теоретический материал для выполнения 1 части 7
1.1.1. Установка и настройка клиента git на своей рабочей станции 7
1.1.2. Создание папки и файлов. Внесение изменений в файл. Создание репозитория 9
1.1.3. Индексация изменений и добавление первого коммита. Проверка состояния репозитория 10
1.1.4. Просмотр истории коммитов 12
1.1.5. Возврат рабочего каталога к одному из предыдущих состояний 13
1.1.6. Создание тегов к коммитам. Просмотр списка тегов 14
1.1.7. Отмена некоторых изменений в рабочем каталоге до индексирования 15
1.1.8. Отмена некоторых изменений в рабочем каталоге после индексирования и до коммита 16
1.1.9. Отмена одного из коммитов в локальном репозитории 17
1.2. Теоретический материал для выполнения 2 части 18
1.2.1. Создание аккаунта на GitHub 18
1.2.2. Создание SSH-ключа для авторизации 20
1.2.3. Добавление связи между репозиторием локальной машины с репозиторием на GitHub при помощи консоли 23
1.2.4. Создание новой ветки в репозитории с помощью команды, внесение изменений, слияние с веткой master 26
1.3. Теоретический материал для выполнения 3 части по вариантам 28
1.3.1. Клонирование репозитория 28
1.3.2. Выгрузка изменений в удаленный репозиторий 29
1.3.3. Вывод в консоль различий между веткой master и другой веткой 29
1.3.4. Работа с отложенными изменениями - git stash 29
1.3.5. Перемещение коммита в другую ветку 31
1.3.6. Добавление изменений в последний коммит 32
1.3.7. Вывод изменений строк файла по коммитам 34
1.4. Порядок выполнения 4 части 35
1.4.1. Сделать форк репозитория в соответствии с Вашим вариантом 35
1.4.2. Клон репозитория на локальную машину 37
1.4.3. Создание двух веток branch1 и branch2 от последнего коммита в

master 37
1.4.4. Внесение 3 коммитов в каждую из веток, которые меняют один и тот же кусочек файла 37
1.4.5. Выполнение слияния ветки branch1 в ветку branch2, разрешив конфликты при этом 37
1.4.6. Выгрузка всех изменений из двух веток в удаленный репозиторий 38
1.4.7. Внесение 3 коммитов в ветку branch1 38
1.4.8. Повторный клон репозитория еще раз в другую директорию 38
1.4.9. Добавление 3 коммитов в новом клоне в ветку branch1 38
1.4.10. Выгрузка всех изменений из нового репозитория в удаленный репозиторий 39
1.4.11. Возврат в старый клон с репозиторием и выгрузка изменения с опцией --force 39
1.4.12. Получение всех изменений в новом репозитории 39
1.5. Практическая часть 39
1.6. Вывод 43
1.7. Контрольные вопросы 43
1.8. Требования к отчету 44
1.9. Список источников для изучения 45
2. СИСТЕМЫ СБОРКИ 46
2.1. Теоретическая часть 46
2.2. Практическая часть 50
2.2.1. Задание на практическую работу 51
2.2.2. Варианты для выполнения работы 51
2.3. Вывод 52
2.4. Контрольные вопросы 52
2.5. Требования к отчету 53
2.6. Список источников для изучения 53
3. DOCKER 55
3.1. Теоретическая часть 55
3.1.1. Образы 56
3.1.2. Изоляция 56
3.1.3. Работа с портами 57
3.1.4. Именованные контейнеры, остановка и удаление 58
3.1.5. Постоянное хранение данных 59
3.1.6. Тома 60

3.1.7. Монтирование директорий и файлов 61
3.1.8. Переменные окружения 62
3.1.9. Dockerfile 62
3.2. Практическая часть 64
3.3. Вывод 67
3.4. Контрольные вопросы 67
3.5. Требования к отчету 68
ЗАКЛЮЧЕНИЕ 69
СПИСОК ИСТОЧНИКОВ ЛИТЕРАТУРЫ 70
ИНФОРМАЦИЯ ОБ АВТОРАХ 71

ВВЕДЕНИЕ

Данный практикум предназначен для студентов, изучающих дисциплину «Технология разработки программных приложений». Он охватывает ключевые аспекты современной разработки программного обеспечения, включая работу с системами контроля версий (Git), автоматизацию задач с помощью Bash-скриптов, сборку проектов с использованием Gradle, контейнеризацию приложений с Docker и управление инфраструктурой с помощью Ansible.

Практические работы направлены на формирование у студентов навыков, необходимых для эффективной работы в команде, автоматизации рутинных процессов и развертывания программных решений в различных средах.

Каждая практическая работа сопровождается методическими указаниями по их выполнению, подробным разбором используемых команд и контрольными вопросами для самопроверки. По каждой работе необходимо оформить отчет, выполнив практическую часть и письменно ответив на контрольные вопросы.

1. ОСНОВЫ РАБОТЫ С GIT

Цель практической работы: сформировать навыки работы с системой контроля версий Git и платформой GitHub для управления кодом в индивидуальных и командных проектах.

1.1. Теоретический материал для выполнения 1 части

1.1.1. Установка и настройка клиента git на своей рабочей станции

В первую очередь, необходимо установить Git на локальную машину.

1. Установка в Linux и Unix.

а) Используйте обычный менеджер пакетов Вашего дистрибутива. Откройте терминал и введите подходящие команды.

б) Если установлена 21 или более ранняя версия Fedora, используйте yum install git.

в) Для 22 и последующих версий Fedora вводите dnf install git.

г) Для дистрибутивов, основанных на Debian, например, Ubuntu, используйте apt-get: sudo apt-get install git.

Полный список команд для различных дистрибутивов можно посмотреть по ссылке: https://git-scm.com/downloads/linux.

2. Установка на macOS.

а) Необходимо скачать Git по ссылке: https://git-scm.com/downloads/mac.

б) Запустить загруженный файл.

в) Система может показать окно с ошибкой, где будет написано, что файл скачан с неавторизованного сайта и инсталлятор не может быть запущен. В таком случае нужно зайти в «Системные настройки» — «Безопасность» (Security and Privacy), в появившемся окне будет сообщение об ошибке и кнопка «Open anyway» («Все равно открыть»), на которую нужно нажать.

г) Система покажет окно, уточняющее запуск установки. Необходимо подтвердить действие.

д) Установщик проведет через все необходимые шаги.

3. Установка в Windows.

Необходимо скачать exe-файл инсталлятора с сайта https://git-scm.com/downloads/win и запустить его. Это Git для Windows, он называется msysGit. Установщик спросит о том, добавлять ли в меню проводника возможность запуска файлов с помощью Git Bash (консольная версия) и GUI

(графическая версия). Подтвердите действие, чтобы далее вести работу через консоль в Git Bash. Остальные пункты можно оставить по умолчанию.

Проверить установленную версию Git можно через терминал с использованием команды git --version. Эта проверка подходит для всех операционных систем.

Для работы с git необходимо указать свое имя и адрес электронной почты, используя следующие команды (где Your Name — Ваш логин, your_email@whatever.com — Ваша почта):

```
$ git config --global user.name "Your Name"
$ git config --global user.email "your_email@whatever.com"
```

Для настройки обработки окончания строк для пользователей Unix/Mac необходимо использовать команды:

```
$ git config --global core.autocrlf input
$ git config --global core.safecrlf warn
```

Для пользователей Windows команды выглядят следующим образом:

```
$ git config --global core.autocrlf true
$ git config --global core.safecrlf warn
```

Во избежание нечитаемых строк необходимо использовать команду:

```
$ git config --global core.quotepath off
```

Для просмотра введенных команд следует использовать команду:

```
$ git config --list
```

Вывод команды должен выглядеть похожим образом (рис. 1.1):

```
user.email=petrova@mirea.ru
user.name=Anna
core.autocrlf=true
core.quotepath=off
core.editor=nano
core.safecrlf=warn
```

Рисунок 1.1. Вывод команды git config –list

1.1.2. Создание папки и файлов. Внесение изменений в файл. Создание репозитория

Для перехода в конкретную директорию можно воспользоваться командой cd с указанием полного пути до нужной директории (change directory — изменить каталог). В зависимости от операционной системы формат команды может меняться. На рис. 1.2 показан формат команды для Git Bash в Windows.

```
Den@DESKTOP-UGJ72FC MINGW64 ~
$ cd '/c/Users/Den/Desktop/1 практика по ТРПП'

Den@DESKTOP-UGJ72FC MINGW64 ~/Desktop/1 практика по ТРПП
$ |
```

Рисунок 1.2. Вывод команды cd

Создание директории происходит через команду mkdir <Название папки>, создание файла — touch <имя файла> (рис. 1.3).

```
Den@DESKTOP-UGJ72FC MINGW64 ~/Desktop/1 практика по ТРПП
$ mkdir Project

Den@DESKTOP-UGJ72FC MINGW64 ~/Desktop/1 практика по ТРПП
$ cd Project

Den@DESKTOP-UGJ72FC MINGW64 ~/Desktop/1 практика по ТРПП/Project
$ touch project.html
```

Рисунок 1.3. Создание директории и файла

Для редактирования файла можно воспользоваться командой nano <имя файла> или редактором vim. Вывод команды nano (открытие редактора) выглядит следующим образом (рис. 1.4):

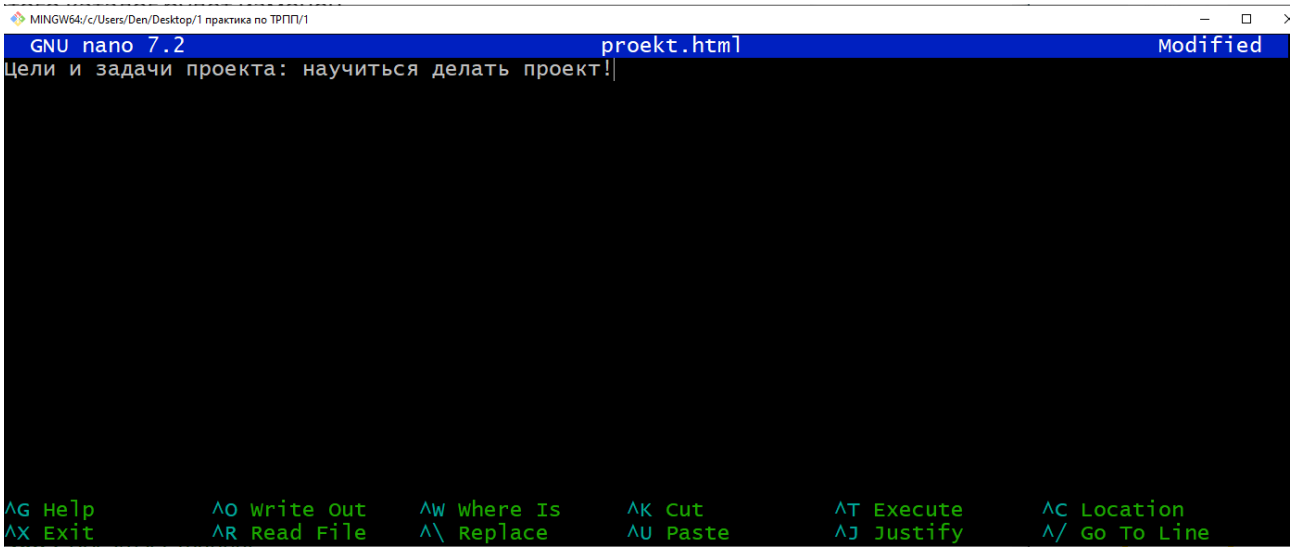

Рисунок 1.4. Редактор nano

После внесения изменений в файл необходимо нажать сочетание клавиш CTRL+x для выхода из редактора, для подтверждения сохранения нажать Y, а далее Enter.

Для создания локального репозитория следует выполнить команду:

```
$ git init
Initialized empty Git repository in C:/Users/User/Project/.git/
```

Локальный репозиторий позволяет работать над локальной версией проекта. Скрытый каталог .git хранит все внутренние данные для отслеживания репозитория.

1.1.3. Индексация изменений и добавление первого коммита. Проверка состояния репозитория

Индексацией изменений в Git называется процесс внесения изменений в индекс (временное хранилище) для последующего добавления в коммит. Можно добавлять и удалять файлы из индекса. Коммит означает сохранение (фиксирование) проиндексированных изменений в репозитории и создание нового «снимка» проекта, который отражает его текущее состояние.

Индексация изменений (git add) и добавление в коммит (git commit) используются на практике следующим образом:

```
$ git add <Название вашего файла>
$ git commit -m "Ваш текст для коммита"
```

Флаг -m позволяет добавить к коммиту комментарий. Это помогает корректно отображать другим разработчикам проекта информацию о том, какие изменения были сделаны.

Если необходимо добавить все измененные и новые файлы в индекс сразу, то используется команда:

```
$ git add --all
```

Если необходимо подготовить к коммиту текущую папку и все файл в ней используется команда:

```
$ git add .
```

Для удобной работы с файлами репозитория необходимо знать, в каком состоянии находятся эти файлы. Для просмотра статуса текущих файлов используется следующая команда:

```
$ git status
```

Каждый файл может находиться в состоянии tracked (отслеживаемый) и untracked (неотслеживаемый). Если Git отслеживает файлы, то может показать, какие из них являются измененными (modified) и подготовленными к коммиту (staged). Git status позволяет продемонстрировать данные состояния (рис. 1.5). В данном случае зеленым помечены изменения, которые добавлены в индекс и готовы к коммиту (changes to be committed) . Статус modified красного цвета отмечает, что изменения были воспроизведены, однако они не добавлены в индекс. Созданный файл 1.txt не отслеживается (untracked files) в репозитории, поскольку так же не добавлен в индекс.

```
Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   pr.html
        new file:   project.html

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   pr.html

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        1.txt
```

Рисунок 1.5. Проверка состояния файлов

В случае, если все изменения во всех файлах проиндексированы и добавлены в коммит, команда git status выведет информацию о том, что в текущем состоянии рабочего каталога нет изменений, ожидающих записи (рис. 1.6):

```
Den@DESKTOP-UGJ72FC MINGW64 ~/Desktop/1 практика по ТРПП/1 (master)
$ git status
On branch master
nothing to commit, working tree clean
```

Рисунок 1.6. Проверка статуса репозитория

1.1.4. Просмотр истории коммитов

Для просмотра истории коммитов используется команда:

```
$ git log
```

Вывод команды позволяет просмотреть хэш коммита, имя автора и дату создания (рис. 1.7):

```
Den@DESKTOP-UGJ72FC MINGW64 ~/Desktop/1 практика по ТРПП/1 (master)
$ git log
commit 844ec5401cd522a626be4afa555b0e29b68d048c (HEAD -> master)
Author: Anna <petrova@mirea.ru>
Date:   Wed Feb 7 20:08:55 2024 +0300

    Во втором файле произведены вторые изменения

commit 171c7a4a2a51c7819fb80376572d56047801a738
Author: Anna <petrova@mirea.ru>
Date:   Wed Feb 7 20:05:56 2024 +0300

    Во втором файле произведены первые изменения

commit eede3e5c506dc32edfe72ea80341ad889e3d804a
Author: Anna <petrova@mirea.ru>
Date:   Wed Feb 7 19:27:13 2024 +0300

    Это мой первый коммит: я добавил новый файл
```

Рисунок 1.7. Вывод истории коммитов

Для обзора коммитов в разных форматах можно воспользоваться следующими вариантами:

```
$ git log --pretty=oneline --max-count=2
$ git log --pretty=oneline --since='5 minutes ago'
$ git log --pretty=oneline --until='5 minutes ago'
$ git log --pretty=oneline --author=<your name>
$ git log --pretty=oneline --all
```

Также можно воспользоваться командой вывода истории коммитов с форматированным выводом:

```
$ git log --pretty=format:"%h %ad | %s%d [%an]" --graph --
date=short
```

Рассмотрим её в деталях:

1. %h — укороченный хэш коммита;
2. %d — дополнения коммита («головы» веток или теги);
3. %ad — дата коммита;
4. %s — комментарий;
5. %an — имя автора;
6. --graph — отображает дерево коммитов в виде ASCII-графика;
7. --date=short — сохраняет формат даты коротким.

1.1.5. Возврат рабочего каталога к одному из предыдущих состояний

Для получения старой версии каталога необходимо узнать для начала хэши коммитов через одну из команд из предыдущей главы. В качестве примера приведена команда git log --pretty=format (рис. 1.8):

```
Den@DESKTOP-UGJ72FC MINGW64 ~/Desktop/1 практика по ТРПП/1 (master)
$ git log --pretty=format:"%h %ad | %s%d [%an]" --graph --date=short
* 844ec54 2024-02-07 | Во втором файле произведены вторые изменения (HEAD -> master) [Anna]
* 171c7a4 2024-02-07 | Во втором файле произведены первые изменения [Anna]
* eede3e5 2024-02-07 | Это мой первый коммит: я добавил новый файл [Anna]
```

Рисунок 1.8. Вывод истории коммитов через git log --pretty=format

Получение старой версии возможно с использованием команды git checkout с указанием конкретного хэша коммита:

```
$ git checkout <Хэш коммита>
```

Результатом выполнения команды с использованием самого первого хэша будет следующий вывод (рис. 1.9):

```
Den@DESKTOP-UGJ72FC MINGW64 ~/Desktop/1 практика по ТРПП/1 (master)
$ git checkout eede3e5
Note: switching to 'eede3e5'.

You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.

If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:

  git switch -c <new-branch-name>

Or undo this operation with:

  git switch -

Turn off this advice by setting config variable advice.detachedHead to false

HEAD is now at eede3e5 Это мой первый коммит: я добавил новый файл

Den@DESKTOP-UGJ72FC MINGW64 ~/Desktop/1 практика по ТРПП/1 ((eede3e5...))
$ cat proekt.html
Цели и задачи проекта: научиться делать проект!
```

Рисунок 1.9. Получение старой версии каталога

После возврата к первому хэшу команда cat отображает первоначальное содержимое в файле.

Для возврата к последнему состоянию необходимо использовать следующую команду:

```
$ git checkout master
Previous HEAD position was eede3e5 Это мой первый коммит: я доба-
вил новый файл
Switched to branch 'master'
```

Ветка master — основная ветка проекта в системе Git. Она появляется сразу после клонирования или инициализации репозитория.

1.1.6. Создание тегов к коммитам. Просмотр списка тегов

Теги необходимы для идентификации коммитов. Можно использовать их для указания коммита, на котором была завершена разработка beta-версии проекта.

Для перехода на конкретный коммит необходимо воспользоваться командой:

```
$ git checkout <Хэш коммита>
```

Создание тега первой версии можно сделать через команду:

```
$ git tag v1
```

Чтобы создать тег для версии, которая идет перед текущей версией, с названием v1-beta, надо переключиться на предыдущую версию. Вместо поиска по хэшу можно использовать ^, обозначающее «родитель v1». При успешном переходе на другую версию команда cat с названием файла покажет содержимое этой версии.

```
$ git checkout v1^
$ cat <Название файла>
```

Теперь можно присвоить тег для версии v1-beta через команду:

```
$ git tag v1-beta
```

Переключаться между версиями можно командами:

```
$ git checkout v1
$ git checkout v1-beta
```

Просмотр тегов возможен с использованием:

```
$ git tag
v1
v1-beta
```

1.1.7. Отмена некоторых изменений в рабочем каталоге до индексирования

В случае если внесены изменения в файл и дальнейшие действия по внесению изменений в индекс не произведены, но необходимо отменить эти изменения, не заходя в редактор вручную, то необходимо воспользоваться командой:

```
$ git checkout <Название файла>
```

Чтобы в рамках практической работы воспользоваться этой командой, необходимо выполнить следующие действия:

1. Убедиться, что текущая ветка проекта — master.
2. Изменить файл в рабочем каталоге при помощи команды nano <имя файла> (внести свои изменения).

3. Проверить состояние рабочего каталога при помощи команды git status. Убедиться, что файл был изменен, но не проиндексирован.

4. Воспользоваться командой git checkout <имя файла>.

5. С использованием команд git status и cat <имя файла> убедиться, что содержимое файла окатилось к предыдущей версии до модификации.

1.1.8. Отмена некоторых изменений в рабочем каталоге после индексирования и до коммита

В случае, если файл изменен и произведена индексация этих изменений, но необходимо откатить содержимое файла обратно, необходимо воспользоваться командами:

```
$ git reset HEAD <Название файла>
$ git checkout <Название файла>
```

В отличие от предыдущей главы, индексация изменений уже произведена, поэтому для ее отмены необходимо воспользоваться git reset HEAD <имя файла>. Команда reset сбрасывает буферную зону к HEAD. Это очищает буферную зону от изменений, которые мы только что проиндексировали. Вывод команды на примере представлен на рис. 1.10:

```
Den@DESKTOP-UGJ72FC MINGW64 ~/Desktop/1 практика по ТРПП/1 (master)
$ nano proekt.html

Den@DESKTOP-UGJ72FC MINGW64 ~/Desktop/1 практика по ТРПП/1 (master)
$ git add proekt.html

Den@DESKTOP-UGJ72FC MINGW64 ~/Desktop/1 практика по ТРПП/1 (master)
$ git status
On branch master
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   proekt.html


Den@DESKTOP-UGJ72FC MINGW64 ~/Desktop/1 практика по ТРПП/1 (master)
$ git reset HEAD proekt.html
Unstaged changes after reset:
M       proekt.html
```

Рисунок 1.10. Отмена индексации файла

Команда reset (по умолчанию) не изменяет рабочий каталог. Поэтому рабочий каталог все еще содержит нежелательный комментарий. Поэтому команда git

checkout <Название файла> позволяет откатить содержимое каталога к предыдущему состоянию (рис. 1.11).

```
Den@DESKTOP-UGJ72FC MINGW64 ~/Desktop/1 практика по ТРПП/1 (master)
$ git checkout proekt.html
Updated 1 path from the index

Den@DESKTOP-UGJ72FC MINGW64 ~/Desktop/1 практика по ТРПП/1 (master)
$ git status
On branch master
nothing to commit, working tree clean
```

Рисунок 1.11. Возврат файла к предыдущей версии

Чтобы в рамках практической работы воспользоваться вышеперечисленными командами, необходимо выполнить следующие действия:

1. Убедиться, что текущая ветка проекта — master.
2. Изменить файл в рабочем каталоге при помощи команды nano <имя файла> (внести свои изменения).
3. Произвести индексацию изменений через git add.
4. Проверить состояние рабочего каталога при помощи команды git status. Убедиться, что файл проиндексирован и готов к коммиту.
5. Воспользоваться командой git reset HEAD <имя файла> для сброса буферной зоны к HEAD.
6. Воспользоваться командой git checkout <имя файла>.
7. С использованием команд git status и cat <имя файла> убедиться, что содержимое файла окатилось к предыдущей версии до модификации.

1.1.9. Отмена одного из коммитов в локальном репозитории

В случае, если файл изменен, проиндексирован и добавлен в коммит, но необходимо отменить этот коммит, можно воспользоваться командой:

```
$ git revert <Хэш коммита>
```

Данная команда создает новый коммит, который отменяет действие одного из предыдущих коммитов. То есть новый коммит появится в истории, а предыдущие коммиты не изменятся.

После выполнения данной команды откроется редактор, который предложит отредактировать коммит или оставить его в неизмененном состоянии. Чтобы не открывать данный редактор, можно воспользоваться параметром --no-edit.

Использование команды представлено на рис. 1.12.

```
Den@DESKTOP-UGJ72FC MINGW64 ~/Desktop/1 практика по ТРПП/1 (master)
$ git revert HEAD --no-edit
[master d33b95f] Revert "Изменение"
 Date: Wed Feb 7 20:45:50 2024 +0300
 1 file changed, 1 deletion(-)

Den@DESKTOP-UGJ72FC MINGW64 ~/Desktop/1 практика по ТРПП/1 (master)
$ git log --pretty=format:"%h %ad| %s%d [%an]" --graph --date=short
* d33b95f 2024-02-07| Revert "Изменение" (HEAD -> master) [Anna]
* 4cc7853 2024-02-07| Изменение [Anna]
* 844ec54 2024-02-07| Во втором файле произведены вторые изменения [Anna]
* 171c7a4 2024-02-07| Во втором файле произведены первые изменения [Anna]
* eede3e5 2024-02-07| Это мой первый коммит: я добавил новый файл [Anna]
```

Рисунок 1.12. Вывод команды git revert

HEAD — указатель на последний коммит в ветке master.

Также следует отметить, что если необходимо сбросить историю коммитов до определенного коммита, можно воспользоваться командами:

```
$ git reset --soft <Хэш коммита>
$ git reset --mixed <Хэш коммита>
$ git reset --hard <Хэш коммита>
```

Вышеперечисленные команды имеют следующие режимы:

а) soft — откатывает историю к нужному коммиту, но не отменяет изменения ни в индексе, ни в рабочей копии;

б) mixed — откатывает историю и сбрасывает индекс, но не изменения в рабочей области (то есть, измененные файлы сохраняются, но не помечаются для дальнейшего коммита);

в) hard — откатывает историю, сбрасывает индекс и изменения в рабочей области.

Примечание: перед использованием git reset --hard рекомендуется создать резервную копию кода.

1.2. Теоретический материал для выполнения 2 части

1.2.1. Создание аккаунта на GitHub

GitHub — веб-сервис, который основан на системе Git. Это социальная сеть для разработчиков, которая помогает удобно вести коллективную разработку IT-проектов. Здесь можно публиковать и редактировать свой код, комментировать чужие наработки, следить за новостями других пользователей.

Если аккаунта на GitHub ранее не было, необходимо зарегистрироваться по ссылке https://github.com/ (рис. 1.13).

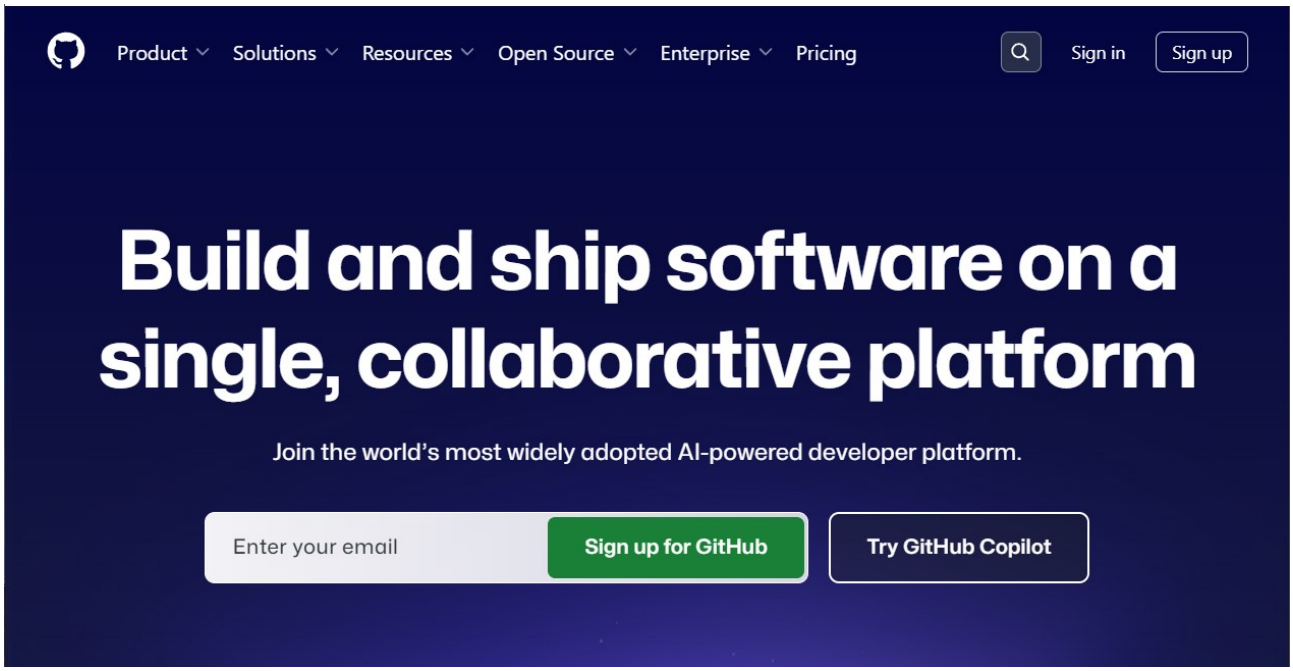

Рисунок 1.13. Стартовая страница GitHub

Для регистрации необходимо нажать на кнопку Sign up, где в открывшейся форме нужно ввести обязательные данные: имя пользователя, адрес электронной почты и пароль. После заполнения полей пройти верификацию (рис. 1.14).

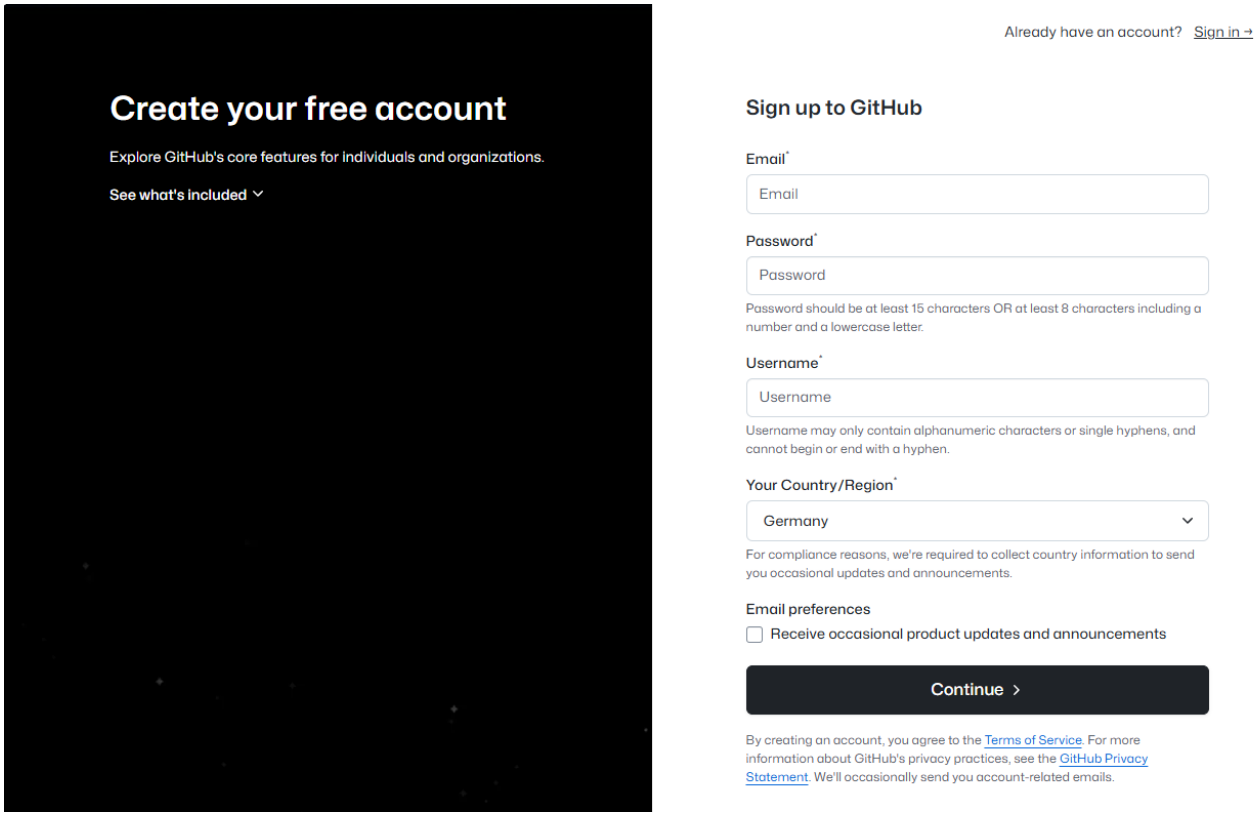

Рисунок 1.14. Шаг регистрации профиля на стартовой странице GitHub

После прохождения всех этапов на сайте, на указанный при регистрации почтовый ящик придет письмо от GitHub. Необходимо подтвердить электронный адрес.

После верификации GitHub предложит создать новый репозиторий, организацию или узнать больше о GitHub. Созданный GitHub-профиль выглядит следующим образом (рис. 1.15):

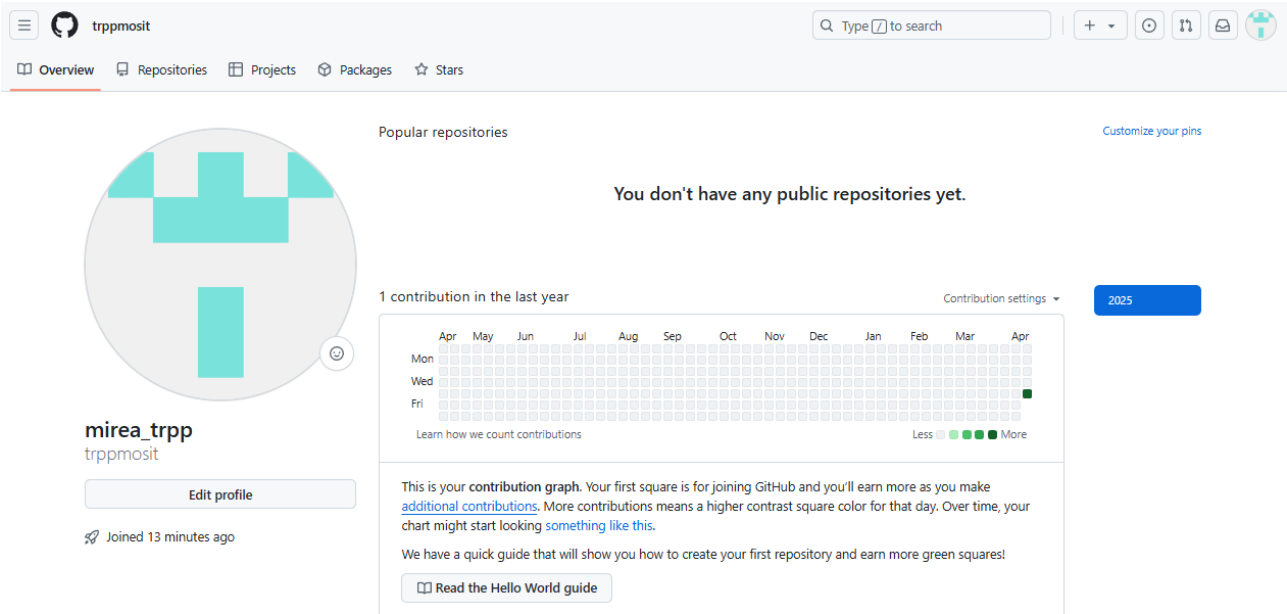

Рисунок 1.15. Профиль после регистрации

1.2.2. Создание SSH-ключа для авторизации

Чтобы работать со своего компьютера с GitHub, иметь доступ к проектам, хранящимся на сервисе, выполнять команды в консоли без постоянного подтверждения пароля, нужно пройти авторизацию у сервера. В этом помогают SSH-ключи.

Каждый SSH-ключ содержит пару: открытый (публичный) и закрытый (приватный) ключ. Открытый ключ отправляется на сервер, и любой человек может использовать его для кодирования информации. Он бесполезен без своей пары — закрытого ключа. Закрытый ключ является секретным и известен только его владельцу. Он расшифровывает данные, зашифрованные соответствующим открытым ключом.

Чтобы пройти авторизацию по SSH-ключу, его надо сгенерировать или найти уже ранее созданный ключ на своем компьютере.

Сначала необходимо проверить, есть ли уже на компьютере ключ. По умолчанию SSH-ключи хранятся в каталоге ~/.ssh, поэтому нужно проверить

содержимое этого каталога. Если SSH-ключ уже существует, то в списке будут файлы с именами вроде id_rsa.pub. Если ключ не создан ранее, то необходимо в каталоге выполнить команду:

```
$ ssh-keygen -t rsa -b 4096 -C "your_mail@example.com"
Generating public/private rsa key pair.
Enter file in which to save the key (/c/Users/User/.ssh/id_rsa):
```

В данной команде:

1. -t rsa — указывает тип ключа (в данном случае RSA),
2. -b 4096 — указывает длину ключа в битах (рекомендуется 4096 для повышенной безопасности),
3. -C "your_email@example.com" — добавляет метку комментария, часто используется для указания email-адреса.

После ввода команды нужно указать расположение файла для сохранения ключа. Если Вы не введете путь до файла и просто нажмете Enter, то ключ сохранится в файле, указанном в скобках.

Теперь нужно установить пароль к Вашему ключу и дважды ввести его. Если Вы не хотите вводить пароль каждый раз, когда используете ключ, пропустите этот шаг, нажав Enter, и ничего не вводите.

Далее необходимо запустить ssh-agent (программу для хранения и управления SSH-ключами) через команду:

```
$ eval "$(ssh-agent -s)"
Agent pid 65404
```

Если в ответ терминал покажет надпись «Agent pid» и число, значит агент запущен. Числа могут быть различными.

Далее нужно добавить ключ с помощью ssh-add ~/.ssh/your_key_name, где указывается верный путь до файла с ключом и его имя. Если название ключа будет id_rsa, то команда будет выглядеть следующим образом:

```
$ ssh-add ~/.ssh/id_rsa
Enter passphrase for /c/Users/User/.ssh/id_rsa:
Identity added: /c/Users/User/.ssh/id_rsa (petrova@mirea.ru)
```

Если Вы устанавливали пароль на ключ, введите его два раза после ввода команды ssh-add (терминал подскажет, когда это сделать).

Для добавления ключа на GitHub, необходимо скопировать содержимое файла с открытым (публичным) ключом. Посмотреть содержимое можно через команду:

```
$ cat ~/.ssh/id_rsa.pub
```

Выведенный ключ необходимо полностью скопировать из терминала. Далее необходимо зайти на страницу работы с ключами через настройки в своем аккаунте GitHub и нажать на кнопку New SSH key (рис. 1.16).

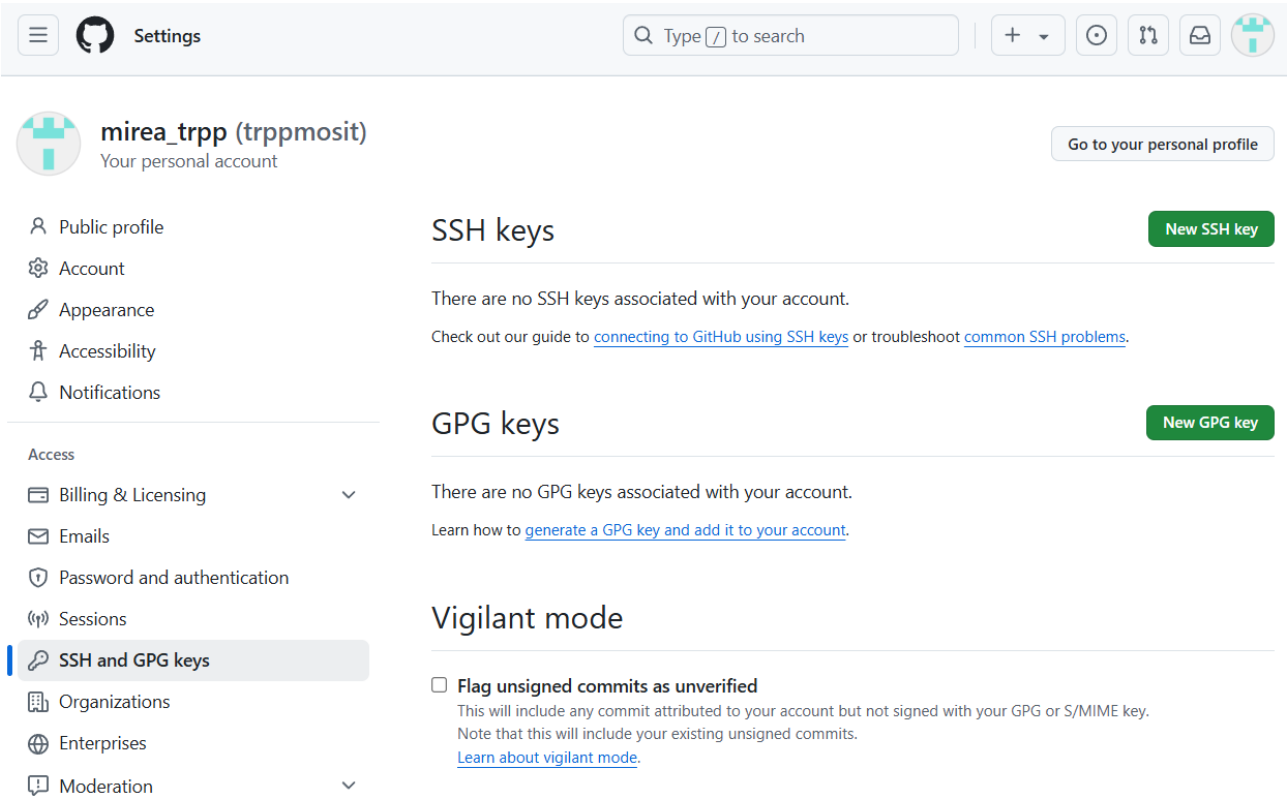

Рисунок 1.16. Страница работы с SSH-ключами

Нажав на New SSH key, в поле Title вводим любое имя ключа, в поле Key — скопированный ключ. Далее нажимаем на кнопку Add SSH key (рис. 1.17).

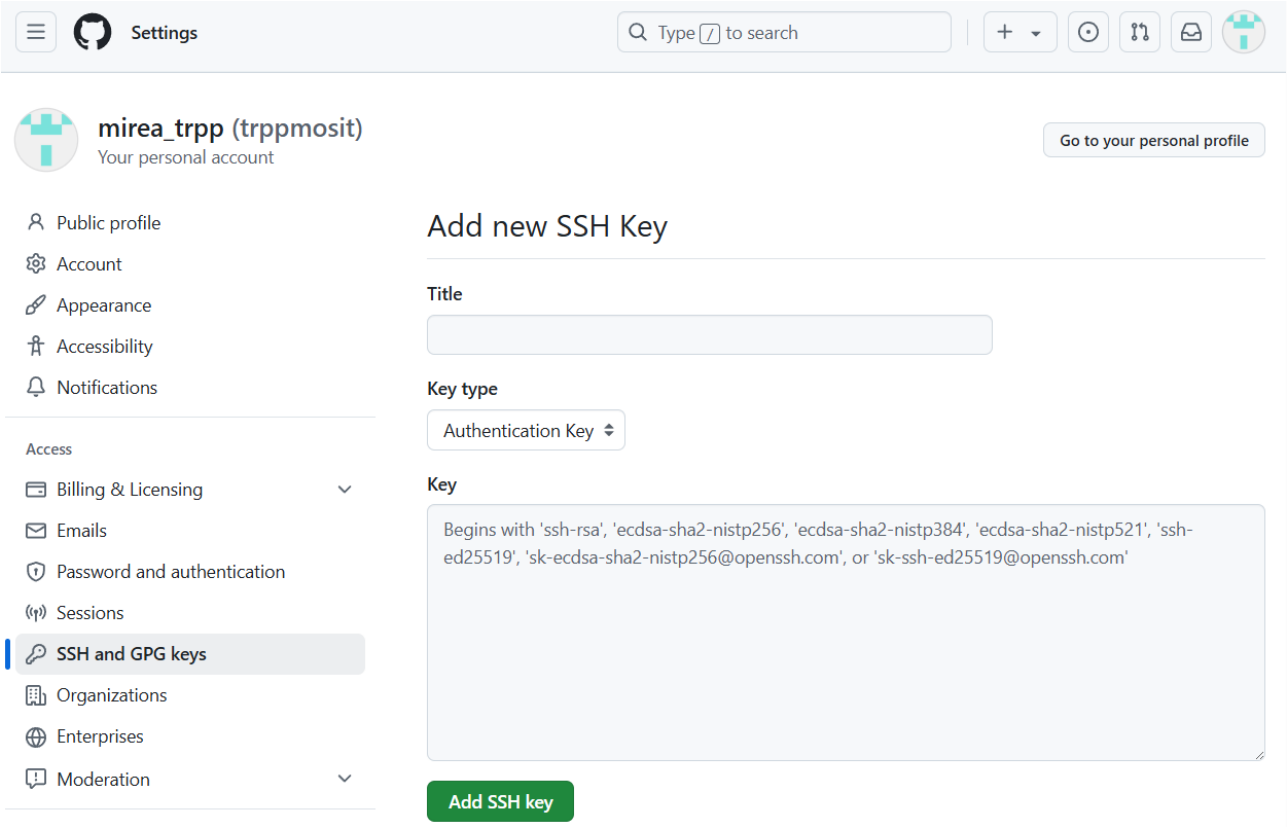

Рисунок 1.17. Страница работы с SSH-ключами

Добавленный ключ выглядит следующим образом (рис. 1.18)

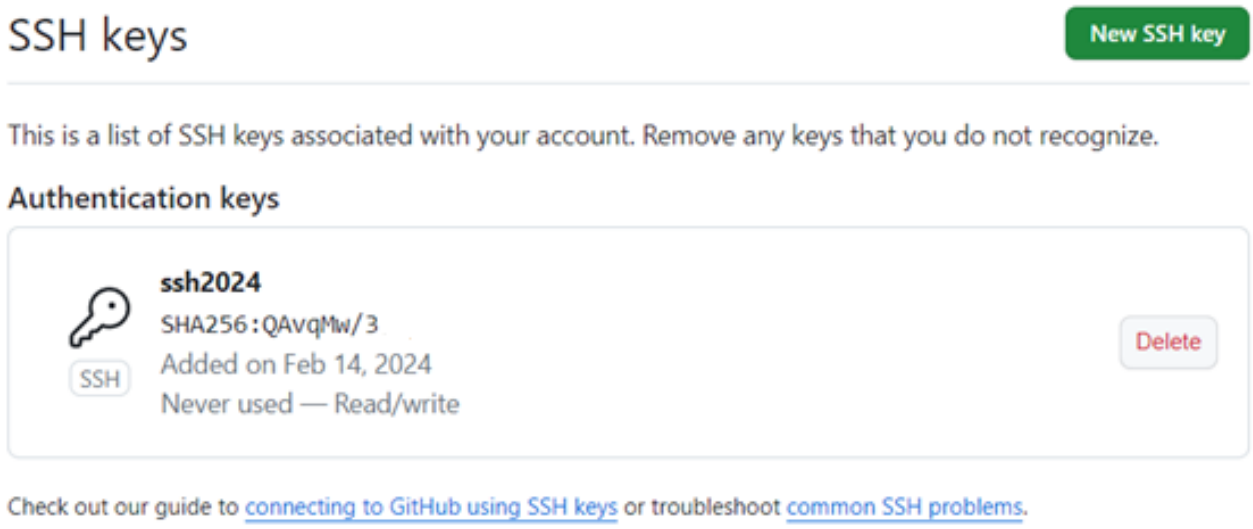

Рисунок 1.18. Добавленный SSH-ключ

1.2.3. Добавление связи между репозиторием локальной машины с репозиторием на GitHub при помощи консоли

Перед тем, как связать локальный и удаленный репозитории, необходимо создать новую папку проекта, добавить в него несколько файлов (например,

readme.txt) и через команду git init создать новый локальный репозиторий (пункт 1.1.2).

Для создания удаленного репозитория на GitHub необходимо зайти во вкладку создания репозитория (рис. 1.19).

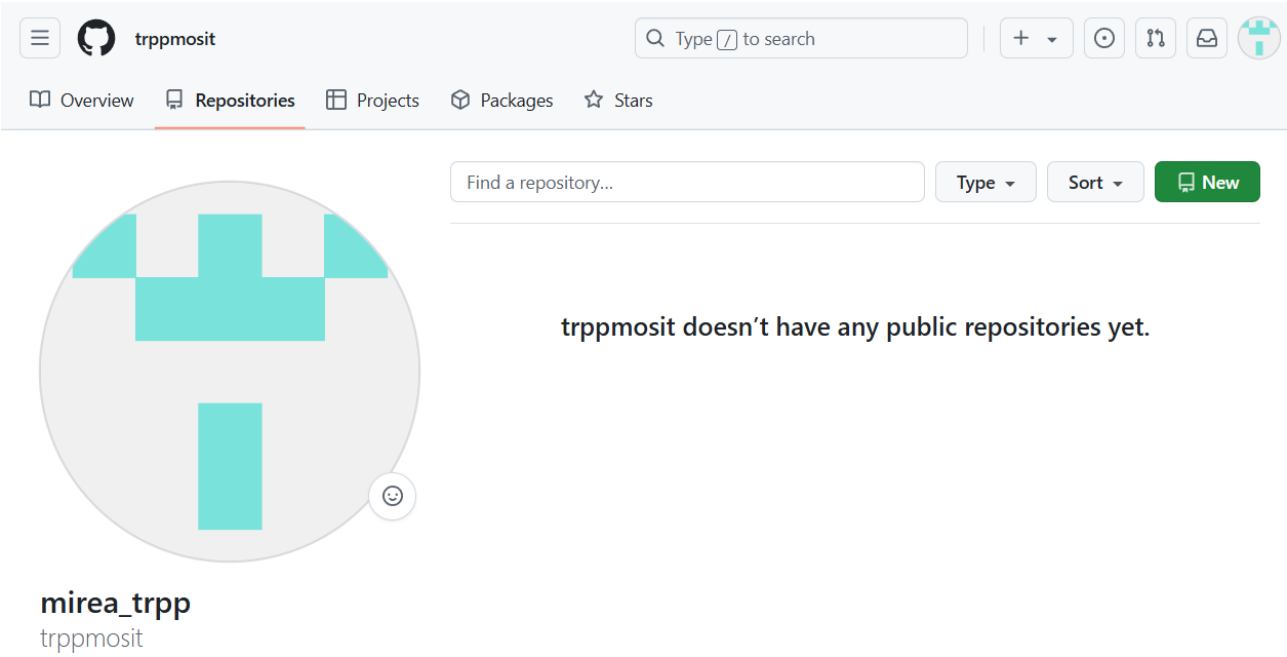

Рисунок 1.19. Вкладка репозиториев

Нажав на кнопку New, в появившемся окне откроется форма добавления нового репозитория. Необходимо дать название, выбрать приватность (публичный или приватный) (рис. 1.20).

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? Import a repository.

Required fields are marked with an asterisk (*).

Owner * Repository name *

trppmosit /

Great repository names are short and memorable. Need inspiration? How about verbose-octo-system ?

Description (optional)

Public
Anyone on the internet can see this repository. You choose who can commit.

Private
You choose who can see and commit to this repository.

Initialize this repository with:

Add a README file
This is where you can write a long description for your project. Learn more about READMEs.

Add .gitignore

.gitignore template: None

Choose which files not to track from a list of templates. Learn more about ignoring files.

Choose a license

License: None

A license tells others what they can and can't do with your code. Learn more about licenses.

You are creating a public repository in your personal account.

Create repository

Рисунок 1.20. Создание репозитория

После проделанных шагов создается пустой репозиторий на GitHub.

Для того, чтобы связать репозиторий на локальной машине и удаленный репозиторий, необходимо использовать URL-ссылку на созданный удаленный репозиторий с использованием SSH-подключения (рис. 1.21).

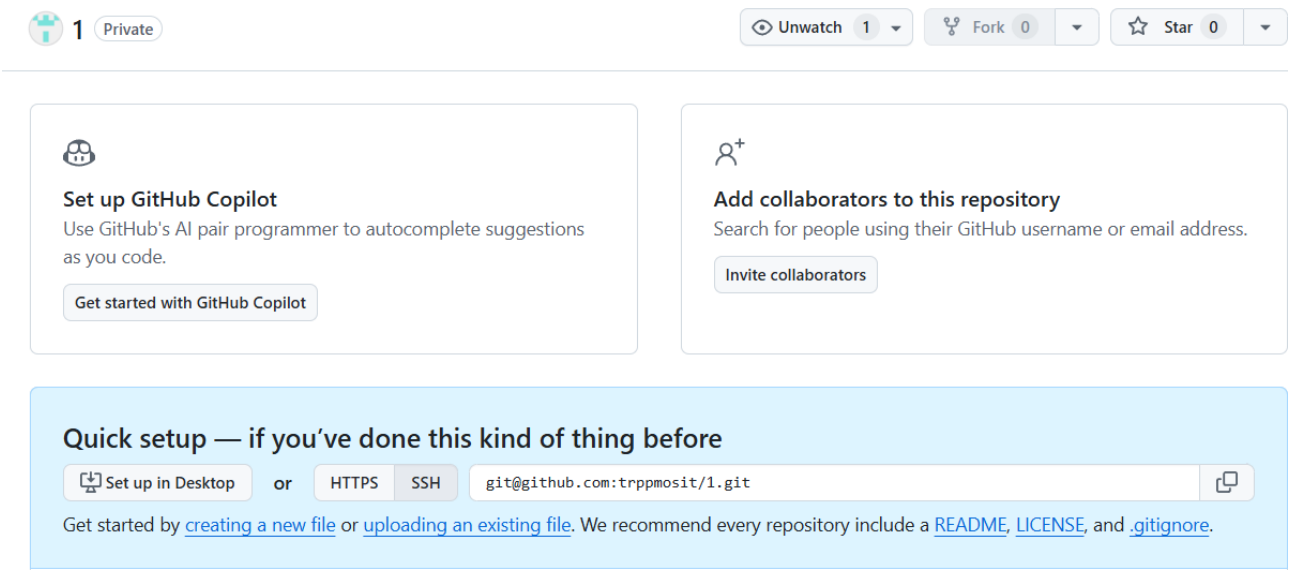

Рисунок 1.21. Копирование URL-ссылки репозитория через SSH

После этого связать репозитории друг с другом можно через команду:

```
$ git remote add origin git@github.com:<Ваши имя и название репо-
зитория>.git
```

1.2.4. Создание новой ветки в репозитории с помощью команды, внесение изменений, слияние с веткой master

Перед тем, как создать новую ветку на локальном репозитории, нужно инициализировать основную ветку master в проекте. Это можно сделать через создание первого коммита.

Для того, чтобы создать ветку, необходимо ввести одну из следующих команд:

```
$ git branch branch1
$ git checkout -b branch1
```

Первая команда создаст ветку с названием branch1, вторая команда создаст ветку и сразу перейдет на нее.

Чтобы переключаться между ветками, необходимо воспользоваться командой:

```
$ git checkout <название ветки>
```

Данная команда перейдет на указанную ветку, если она была создана. Также можно переключаться на ветку master:

```
$ git checkout master
```

В рамках 2 части практической необходимо, чтобы были созданы ветки master и branch1. В файлах, которые были добавлены в локальный репозиторий на предыдущем этапе, необходимо произвести изменения и закоммитить их (рис. 1.22).

```
Den@DESKTOP-UGJ72FC MINGW64 ~/Desktop/2 часть практики по ТРПП (branch1)
$ nano file1.txt

Den@DESKTOP-UGJ72FC MINGW64 ~/Desktop/2 часть практики по ТРПП (branch1)
$ nano readme.txt

Den@DESKTOP-UGJ72FC MINGW64 ~/Desktop/2 часть практики по ТРПП (branch1)
$ git add .
warning: in the working copy of 'file1.txt', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'readme.txt', LF will be replaced by CRLF the next time Git touches it

Den@DESKTOP-UGJ72FC MINGW64 ~/Desktop/2 часть практики по ТРПП (branch1)
$ git add .

Den@DESKTOP-UGJ72FC MINGW64 ~/Desktop/2 часть практики по ТРПП (branch1)
$ git commit -m "Внесли изменения в ветке branch1"
[branch1 ca64107] Внесли изменения в ветке branch1
 2 files changed, 2 insertions(+), 2 deletions(-)
```

Рисунок 1.22. Внесение изменений в файл, добавление в индекс и коммит

Слияние производится с помощью следующей команды:

```
$ git merge <название ветки>
```

Название ветки указывает на то, из какой ветки необходимо слить изменения. Например, если мы находимся в ветке branch2 и нам надо произвести слияние из ветки branch1 в branch2 (именно в таком порядке). То есть нам нужно, чтобы все изменения из ветки branch1 оказались и в ветке branch2. Тогда для этого мы должны, находясь в ветке branch2, ввести команду git merge branch1. В этом случае мы выполним поставленную задачу. Выполнение задания по слиянию ветки master с веткой branch1 выглядит следующим образом (рис. 1.23):

```
$ git merge branch1
Updating 27fcbf0..ca64107
Fast-forward
 file1.txt  | 2 +-
 readme.txt | 2 +-
 2 files changed, 2 insertions(+), 2 deletions(-)
```

Рисунок 1.23. Слияние веток

В данном случае все изменения, которые были сделаны в файлах ветки branch1, были добавлены в ветку master.

1.3. Теоретический материал для выполнения 3 части по вариантам

1.3.1. Клонирование репозитория

Клонирование репозитория на локальную машину можно выполнить при помощи команды:

```
$ git clone <ссылка на репозиторий>
```

В качестве репозитория можно использовать любой публичный репозиторий. Ссылку для клонирования можно получить на странице самого репозитория по выпадающему меню Code (рис. 1.24).

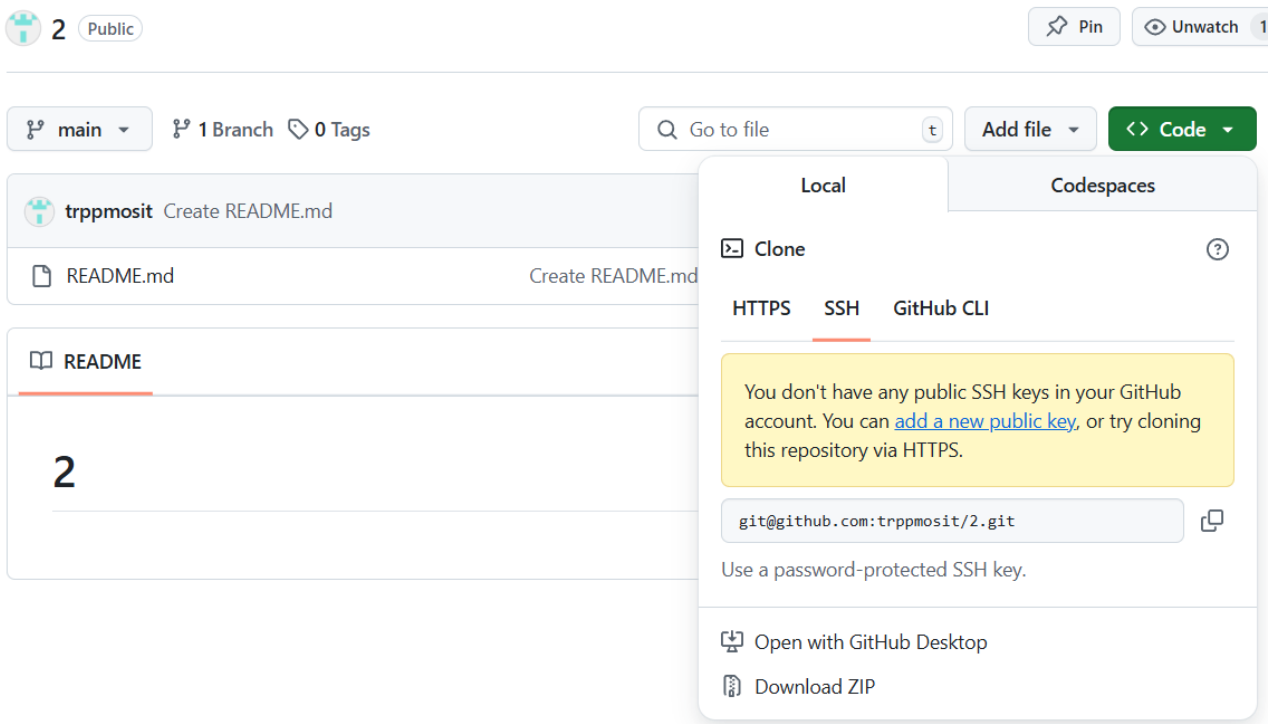

Рисунок 1.24. Ссылка на репозиторий для последующего клонирования

Примечание: клонировать репозиторий необходимо в отдельный каталог, вне текущего репозитория.

Успешное клонирование продемонстрирует следующий результат (рис. 1.25):

```
Cloning into 'part2'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
Receiving objects: 100% (3/3), done.
```

Рисунок 1.25. Результат клонирования

1.3.2. Выгрузка изменений в удаленный репозиторий

Выгрузка в удаленный репозиторий выполняется с помощью команды:

```
$ git push origin <название ветки>
```

Например, git push origin branch1 выгрузит данные из ветки branch1 локального репозитория в удаленный репозиторий (в нашем случае, из компьютера на GitHub).

1.3.3. Вывод в консоль различий между веткой master и другой веткой

Вывод различий между несколькими вариантами состояния репозитория, в том числе между несколькими ветками, можно вывести при помощи команды git diff. Для вывода различий между двумя ветками можно использовать данную команду в следующем варианте:

```
$ git diff <branch1> <branch2>
```

К примеру, если вывести сравнения между веткой master (она же main) и branch2, то результат будет следующим (рис. 1.26):

```
$ git diff main branch2
diff --git a/1 b/1
index 3a1b1c1..d00491f 100644
--- a/1
+++ b/1
@@ -1,4 +1 @@
 1
-Еще цифра 1
-Еще и еще одна цифра 1
-Еще и еще и еще одна цифра 1
```

Рисунок 1.26. Результат вывода различий

1.3.4. Работа с отложенными изменениями - git stash

Git имеет возможность отложить изменения, записав их в специальное хранилище. Сделать это можно при помощи команды git stash. При использовании данной команды изменения в файлах записываются в условный «архив», который

будет хранить их без индексирования и коммита. Такой функционал может пригодиться, если есть необходимость перейти к работе с другой веткой.

Для работы с данной командой сначала нужно внести какое-нибудь изменение в условный файл project.html через редактор (рис. 1.27).

```
  GNU nano 7.2                    project.html
это изменение для проверки stash

                          [ Read 1 line ]
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify
```

Рисунок 1.27. Внесение изменений в файл

Запомнить эти изменения можно с использованием команды:

```
$ git stash
Saved working directory and index state WIP on master: fa40954 2
```

Результат команды означает, что команда git stash сохранила неподтвержденные изменения в рабочем каталоге и индексе с пометкой WIP (незавершенная работа) на ветке master и с идентификатором коммита fa40954. Изменения записались в архив и пропали из текущей версии (рис. 1.28).

```
  GNU nano 7.2                    project.html
1

^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify
```

Рисунок 1.28. Результат использования git stash

Теперь нужно добавить изменения, сохраненные при помощи git stash в ветку newbranch. Для этого необходимо воспользоваться командой git stash pop.

Если ветка ранее не была создана, то необходимо использовать команду git branch <название ветки> для ее инициализации. После перехода на новую ветку необходимо выполнить команду:

```
$ git stash pop
```

В результате команда позволит применить сохраненные изменения в новой ветке и одновременно удалить их из списка сохранений. Через команду cat видно, что изменения файла project.html применились для новой ветки (рис. 1.29):

```
Den@DESKTOP-UGJ72FC MINGW64 ~/Desktop/1 практика по ТРПП/Project (master)
$ git checkout newbranch
Switched to branch 'newbranch'

Den@DESKTOP-UGJ72FC MINGW64 ~/Desktop/1 практика по ТРПП/Project (newbranch)
$ git stash pop
On branch newbranch
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   1.txt
        modified:   project.html

no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@{0} (97a5edc49f3329bff3eca966bd549eed2f1a7cb0)

Den@DESKTOP-UGJ72FC MINGW64 ~/Desktop/1 практика по ТРПП/Project (newbranch)
$ cat project.html
это изменение для проверки stash
```

Рисунок 1.29. Результат использования git stash pop

1.3.5. Перемещение коммита в другую ветку

Перемещать коммиты между ветками можно при помощи команды:

```
$ git cherry-pick
```

Данный функционал можно использовать, если нет необходимости сливать всю ветку целиком, а достаточно взять из нее несколько изменений. Для начала необходимо внести изменения в файл (условно cherry-pick.html) (рис. 1.30):

```
Den@DESKTOP-UGJ72FC MINGW64 ~/Desktop/1 практика по ТРПП/Project (newbranch)
$ cat cherry-pick.html
Изменение для проверки cherry-pick
```

Рисунок 1.30. Внесение изменений в файл

Далее проиндексируем и закоммитим данное изменение. При помощи команды git log узнаем хэш сделанного коммита (рис. 1.31).

```
$ git log
commit ee44b42993b9885ded59e54d725863d2d3e03d56 (HEAD -> newbranch)
Author: Anna <petrova@mirea.ru>
Date:   Fri Apr 18 13:57:46 2025 +0300

    Проверяем cherry-pick
```

Рисунок 1.31. Коммит для изменения

Далее необходимо перейти на ветку master и выполнить команду git cherry-pick для добавления последнего коммита из ветки newbranch. В результате будет получено следующее сообщение (рис. 1.32):

```
Den@DESKTOP-UGJ72FC MINGW64 ~/Desktop/1 практика по ТРПП/Project (newbranch)
$ git checkout master
Switched to branch 'master'

Den@DESKTOP-UGJ72FC MINGW64 ~/Desktop/1 практика по ТРПП/Project (master)
$ git cherry-pick ee44b42993b9885ded59e54d725863d2d3e03d56
[master 733a62e] Проверяем cherry-pick
 Date: Fri Apr 18 13:57:46 2025 +0300
 1 file changed, 1 insertion(+)
 create mode 100644 cherry-pick.html

Den@DESKTOP-UGJ72FC MINGW64 ~/Desktop/1 практика по ТРПП/Project (master)
$ git log
commit 733a62e59a178a1b9310435fcdefa2b6d2fecf2a (HEAD -> master)
Author: Anna <petrova@mirea.ru>
Date:   Fri Apr 18 13:57:46 2025 +0300

    Проверяем cherry-pick
```

Рисунок 1.32. Результат команды cherry-pick

1.3.6. Добавление изменений в последний коммит

Внести изменения в последний коммит, не создавая новый, можно при помощи команды:

```
$ git commit --amend
```

Для проверки работы команды необходимо добавить файл readme.md (рис. 1.33).

```
Den@DESKTOP-UGJ72FC MINGW64 ~/Desktop/1 практика по ТРПП/Project (master)
$ cat readme.md
Тестируем команду ammend
```

Рисунок 1.33. Проверка содержимого readme.md

Теперь добавим файл readme.md в последний коммит при помощи команды git commit --amend. После вызова данной команды откроется текстовый редактор для переименования коммита (рис. 1.34).

Примечание: для добавления файлов в коммит без изменения комментария необходимо добавить флаг --no-edit.

```
    C:/Users/Den/Desktop/1 практика по ТРПП/Project/.git/COMMIT_EDITMSG
Проверяем cherry-pick

# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# Date:      Fri Apr 18 13:57:46 2025 +0300
#
# On branch master
# Changes to be committed:
#       new file:   cherry-pick.html
#
# Untracked files:
#       cherry-pick
#
#
```

Рисунок 1.34. Текстовый редактор для переименования коммита

Результат переименования коммита должен выглядеть следующим образом (рис. 1.35):

```
Den@DESKTOP-UGJ72FC MINGW64 ~/Desktop/1 практика по ТРПП/Project (master)
$ git commit --amend
[master 2a228c9] Добавили readme.md
 Date: Fri Apr 18 13:57:46 2025 +0300
 1 file changed, 1 insertion(+)
 create mode 100644 cherry-pick.html

Den@DESKTOP-UGJ72FC MINGW64 ~/Desktop/1 практика по ТРПП/Project (master)
$ git log
commit 2a228c986a6e7cd32e88a6d22e27ce3597fdde23 (HEAD -> master)
Author: Anna <petrova@mirea.ru>
Date:   Fri Apr 18 13:57:46 2025 +0300

    Добавили readme.md
```

Рисунок 1.35. Измененный коммит

1.3.7. Вывод изменений строк файла по коммитам

Существует несколько вариантов того, как можно вывести информацию об изменениях строк файла. Первый вариант — использование следующей команды:

```
$ git blame <название файла>
```

В качестве примера воспользуемся данной командой для файла project.html (рис. 1.36):

```
Den@DESKTOP-UGJ72FC MINGW64 ~/Desktop/1 практика по ТРПП/Project (master)
$ git blame project.html
f5817b6e (Anna 2025-04-18 13:45:15 +0300 1) <<<<<<< HEAD
^6a29358 (Anna 2025-04-17 13:46:09 +0300 2) 1
f5817b6e (Anna 2025-04-18 13:45:15 +0300 3) =======
f5817b6e (Anna 2025-04-18 13:45:15 +0300 4) Изменение для проверки cherry-pick
f5817b6e (Anna 2025-04-18 13:45:15 +0300 5) >>>>>>> 8a8331e (Проверка cherry-pick)
```

Рисунок 1.36. Результат выполнения git blame

Второй вариант — использование команды:

```
$ git log -p <название файла>
```

В результате мы получим сообщение для каждого изменения в коммитах (рис. 1.37):

```
Den@DESKTOP-UGJ72FC MINGW64 ~/Desktop/1 практика по ТРПП/Project (master)
$ git log -p project.html
commit f5817b6ea051b18ababbb58d5dd875e5b34329de
Author: Anna <petrova@mirea.ru>
Date:   Fri Apr 18 13:45:15 2025 +0300

    cherry-pick

diff --git a/project.html b/project.html
index d00491f..03e5b71 100644
--- a/project.html
+++ b/project.html
@@ -1 +1,5 @@
+<<<<<<< HEAD
 1
+=======
+Изменение для проверки cherry-pick
+>>>>>>> 8a8331e (Проверка cherry-pick)
```

Рисунок 1.37. Результат выполнения git log -p

1.4. Порядок выполнения 4 части

В данной части расписан порядок выполнения 4 части практической работы с упоминанием команд, описанных в главах ранее.

1.4.1. Сделать форк репозитория в соответствии с Вашим вариантом.

Форк репозитория — это личная копия чужого репозитория, которая хранится в учетной записи на GitHub.

Для выполнения задания необходимо выполнить следующие действия:

1. Заходим на GitHub под своим логином и паролем.
2. Переходим по ссылке в репозиторий, который собираемся форкать.
3. В верхнем правом углу нажимаем кнопку Fork (рис. 1.38).

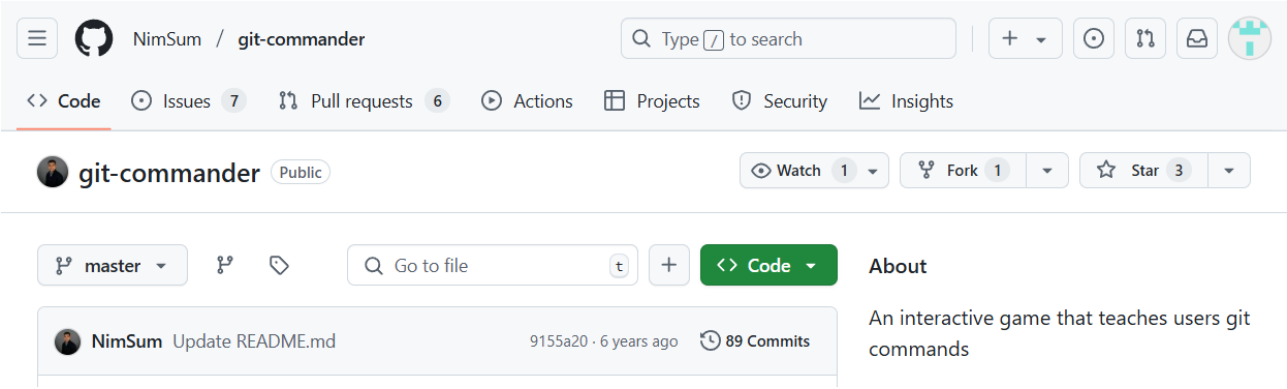

Рисунок 1.38. Кнопка Fork

Далее необходимо подтвердить создание форка (рис. 1.39):

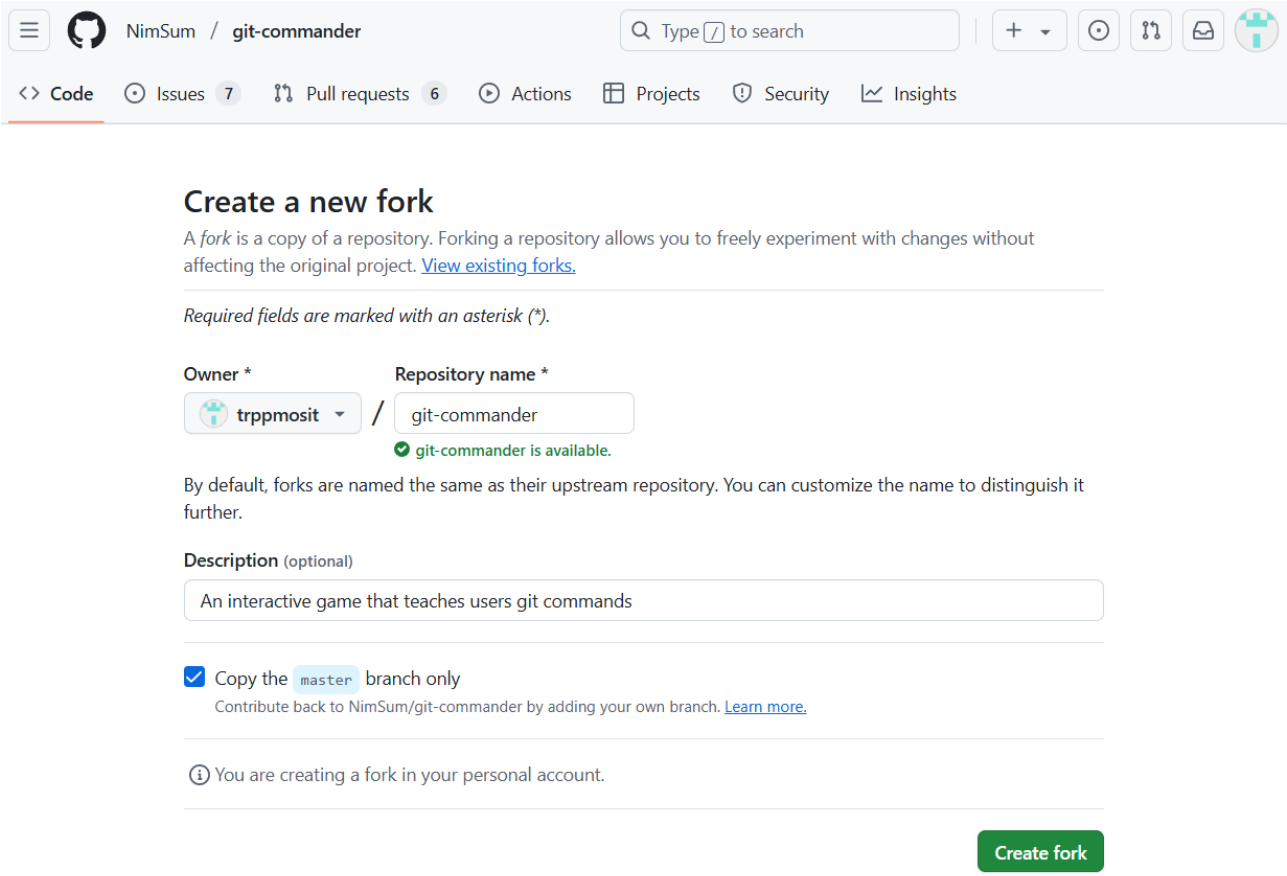

Рисунок 1.39. Кнопка Create fork

После этого копия чужого репозитория появится в Вашем аккаунте (рис. 1.40):

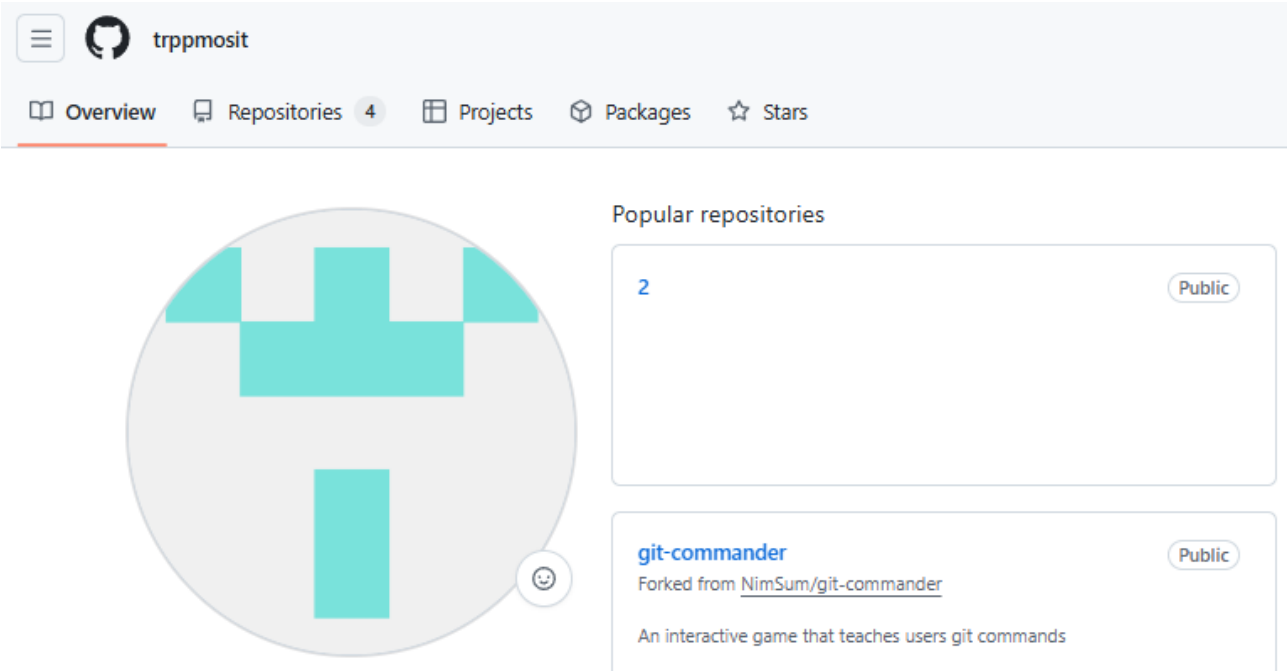

Рисунок 1.40. Результат выполнения форка репозитория

1.4.2. Клон репозитория на локальную машину

Для выполнения задания необходимо выполнить следующие действия:

1. Перейдите в другую папку, не вложенную в предыдущие репозитории.
2. Склонируйте репозиторий в эту папку (пункт 1.3.1).

1.4.3. Создание двух веток branch1 и branch2 от последнего коммита в master

Для создания веток необходимо воспользоваться пунктом 1.2.4.

1.4.4. Внесение 3 коммитов в каждую из веток, которые меняют один и тот же кусочек файла

Примечание: файлы должны назваться одним именем в двух разных ветках.

Для добавления коммитов необходимо воспользоваться пунктом 1.1.3.

1.4.5. Выполнение слияния ветки branch1 в ветку branch2, разрешив конфликты при этом

Для слияния веток необходимо использовать команду git merge (пункт 1.2.4). При слиянии должен появиться конфликт.

Примечание: конфликт должен произойти обязательно.

Существует несколько вариантов разрешения конфликтов:

1. Вручную разрешить конфликты.

После merge в файле появятся куски из обеих сливаемых версий (рис. 1.41):

```
  GNU nano 7.2                                    1.txt
<<<<<<< HEAD
uy
uy
uy
=======
Yt
Yt
Yt
>>>>>>> branch1
```

Рисунок 1.41. Конфликт при слиянии

Нужно будет вручную отредактировать конфликтный файл (или файлы), при этом не забывая удалить метки, оставленные Git (>>>>>>> branch1). Затем ввести команды:

```
$ git add conflicting-file-name.txt
$ git commit -m "merged A and B"
```

2. Выбрать одну из версий файла.

Можно явным образом указать, какой файл выбирать. Подходит, только если одна из версий не нужна. Это возможно с использованием следующих команд:

```
$ git checkout --ours a.txt
$ git checkout --theirs a.txt
$ git add a.txt
$ git commit -m "added theirs"
```

Для решения конфликта можно воспользоваться командой:

```
$ git mergetool
```

1.4.6. Выгрузка всех изменений из двух веток в удаленный репозиторий

Для выполнения задания необходимо выполнить следующие действия:

1. Для перехода по веткам используйте команду git checkout (пункт 1.2.4).
2. Для выгрузки изменений используйте команду git push (пункт 1.3.2).

1.4.7. Внесение 3 коммитов в ветку branch1

Для добавления коммитов необходимо воспользоваться пунктом 1.1.3.

1.4.8. Повторный клон репозитория еще раз в другую директорию

Для выполнения задания необходимо выполнить следующие действия:

1. Перейдите в другую папку, не вложенную в текущий репозиторий.
2. Склонируйте репозиторий в эту папку (пункт 1.3.1).

1.4.9. Добавление 3 коммитов в новом клоне в ветку branch1

Для добавления коммитов необходимо воспользоваться пунктом 1.1.3.

1.4.10. Выгрузка всех изменений из нового репозитория в удаленный репозиторий

Для выгрузки изменений используйте команду git push (пункт 1.3.2).

1.4.11. Возврат в старый клон с репозиторием и выгрузка изменения с опцией --force

Для выполнения задания необходимо выполнить следующие действия:

1. Перейдите в старый клон.
2. Для выгрузки изменений используйте команду git push (пункт 1.3.2).

Опция --force позволяет произвести принудительную выгрузку в репозиторий, даже если в нем обнаружены конфликты слияния.

1.4.12. Получение всех изменений в новом репозитории

Для выполнения задания необходимо выполнить следующие действия:

1. Перейдите в новый клон.
2. Получите все изменения из удаленного репозитория с использованием команды:

```
$ git pull
```

3. Проверьте результат.

Примечание: при возникновении конфликтов необходимо их исправить.

1.5. Практическая часть

Практическая работа направлена на выполнение 4 частей работы.

Часть 1. Основные команды Git.

1. Установите и настройте клиент git на своей рабочей станции.
2. Создайте локальный репозиторий и добавьте в него несколько файлов.
3. Внесите изменения в один из файлов. Проиндексируйте изменения и проверьте состояние. Сделайте коммит того, что было проиндексировано в репозиторий. Добавьте к коммиту комментарий.
4. Измените еще один файл. Добавьте это изменение в индекс git. Измените файл еще раз. Проверьте состояние и произведите коммит проиндексированного изменения. Теперь добавьте второе изменение в индекс, а затем проверьте состояние с помощью команды git status. Сделайте коммит второго изменения.

5. Просмотрите историю коммитов с помощью команды git log. Ознакомьтесь с параметрами команды и используйте некоторые из них для различного формата отображения истории коммитов.

6. Верните рабочий каталог к одному из предыдущих состояний.

7. Создайте теги к коммитам. Посмотрите список тегов.

8. Отмените некоторые изменения в рабочем каталоге до индексирования.

9. Отмените некоторые изменения в рабочем каталоге после индексирования.

10. Отмените один из коммитов в локальном репозитории.

Часть 2. Работа с GitHub.

1. Создайте аккаунт на GitHub (если отсутствует).

2. Создайте SSH-ключ для авторизации.

3. Создайте репозиторий на GitHub и на локальной машине.

4. Свяжите репозиторий локальной машины с репозиторием на GitHub при помощи консоли.

5. Создайте несколько файлов на локальной машине при помощи консоли.

6. Создайте новую ветку в репозитории с помощью команды, произведите в ней какие-нибудь изменения, а после слейте с веткой master.

Часть 3. Выполнение задания по вариантам (табл. 1.1).

Таблица 1.1. Распределение вариантов

Вариант	Задание
1	1. Клонируйте непустой удаленный репозиторий на локальную машину. 2. Создайте новую ветку и выведите список всех веток. 3. Произведите 3 коммита в новой ветке. 4. Выгрузите изменения в удаленный репозиторий. 5. Откатите ветку обратно на 2 коммита (в том числе в удаленном репозитории). 6. Выведите в консоли различия между веткой master и новой веткой. 7. Слейте новую ветку с master при помощи merge.
2	1. Клонируйте непустой удаленный репозиторий на локальную машину. 2. Создайте тег указывающий на последний коммит в ветке master. 3. Создайте новую ветку и выведите список всех веток. 4. Произведите 3 коммита в новой ветке. 5. Выгрузите все изменения в удаленный репозиторий. 6. Откатите ветку к созданному тегу (в том числе в удаленном репозитории).

Продолжение таблицы 1.1

	7. Выведите в консоли различия между веткой master и новой веткой.
3	1. Клонируйте непустой удаленный репозиторий на локальную машину. 2. Создайте новую ветку и выведите список всех веток. 3. Произведите 3 коммита в новой ветке в новых файлах. 4. Выгрузите изменения в удаленный репозиторий. 5. Произведите какие-нибудь изменения в файле который существует в ветке master, но не коммитьте их. 6. Выполните git stash, переключитесь на ветку master, выполните git stash pop. 9. Выведите в консоли различия между веткой master и новой веткой. 10. Слейте новую ветку с master при помощи merge.
4	1. Клонируйте непустой удаленный репозиторий на локальную машину. 2. Создайте новую ветку и выведите список всех веток. 3. Произведите 3 коммита в новой ветке в разные файлы. 4. Выгрузите изменения в удаленный репозиторий. 5. Добейтесь того, чтобы эти три изменения были представлены одним коммитом. 6. Выведите в консоли различия между веткой master и новой веткой. 7. Слейте новую ветку с master при помощи merge.
5	1. Клонируйте непустой удаленный репозиторий на локальную машину. 2. Создайте две новые ветки и выведите список всех веток 3. Произведите 3 коммита в первой ветке. 4. Переместите (cherry pick) первый коммит во вторую ветку. 5. Выгрузите изменения в удаленный репозиторий. 6. Выведите в консоли различия между веткой master и новыми ветками. 7. Слейте обе ветки с master при помощи merge.
6	1. Клонируйте непустой удаленный репозиторий на локальную машину. 2. Создайте новую ветку и выведите список всех веток. 3. Произведите коммит в ветке master. 4. Произведите 3 коммита в новой ветке в разные файлы. 5. Выгрузите изменения в удаленный репозиторий. 6. Откатите ветку обратно на 2 коммита (в том числе в удаленном репозитории). 7. Выведите в консоли различия между веткой master и новой веткой. 8. Перебазируйте новую ветку на master.
7	1. Клонируйте непустой удаленный репозиторий на локальную машину. 2. Создайте новую ветку и выведите список всех веток. 3. Произведите 3 коммита в новой ветке в разные файлы. 4. Выгрузите изменения в удаленный репозиторий. 5. Произведите revert предпоследнего коммита в новой ветке. 6. Выведите в консоли различия между веткой master и новой веткой. 7. Слейте новую ветку с master при помощи merge.

Окончание таблицы 1.1

8	1. Клонируйте непустой удаленный репозиторий на локальную машину. 2. Создайте две новых ветки и выведите список всех веток. 3. Произведите по 2 коммита в обоих ветках в новых файлах. 4. Выгрузите изменения в удаленный репозиторий. 6. Выведите в консоли различия между веткой master и обоими ветками. 7. Слейте одну ветку с master при помощи merge. 8. Подлейте master ко второй ветке. 9. Выведите в консоли различия между веткой master и обоими ветками.
9	1. Клонируйте непустой удаленный репозиторий на локальную машину. 2. Создайте новую ветку и выведите список всех веток. 3. Произведите 5 коммитов в новой ветке в разные файлы. 4. Выгрузите изменения в удаленный репозиторий. 5. Произведите revert 2-го и 4-го коммита в новой ветке. 6. Выведите в консоли различия между веткой master и новой веткой. 7. Слейте новую ветку с master при помощи merge.
10	1. Клонируйте непустой удаленный репозиторий на локальную машину. 2. Создайте новую ветку и выведите список всех веток. 3. Произведите 3 коммита в новой ветке в разные файлы. 4. Выгрузите изменения в удаленный репозиторий. 5. Произведите какие-нибудь изменения в файле который существует в ветке, но не коммитьте их. 6. Внесите эти изменения в последний коммит (amend). 7. Выведите в консоли различия между веткой master и новой веткой. 8. Слейте новую ветку с master при помощи merge.

Часть 4. Работа с ветвлением кода и решением конфликтов.

1. Выбрать репозиторий согласно варианту (табл. 1.2).

Таблица 1.2. Распределение вариантов

Вариант	Репозиторий
1	https://github.com/liamg/traitor
2	https://github.com/google/model_search
3	https://github.com/gto76/python-cheatsheet
4	https://github.com/ripienaar/free-for-dev
5	https://github.com/pystardust/ytfzf
6	https://github.com/airbnb/javascript
7	https://github.com/kettanaito/naming-cheatsheet
8	https://github.com/Sairyss/domain-driven-hexagon
9	https://github.com/react-native-camera/react-native-camera
10	https://github.com/spring-projects/spring-petclinic

2. Сделайте форк репозитория в соответствии с Вашим вариантом.

3. Склонируйте его на локальную машину.

4. Создайте две ветки branch1 и branch2 от последнего коммита в master.

5. Проведите по 3 коммита в каждую из веток, которые меняют один и тот же кусочек файла.

6. Выполните слияние ветки branch1 в ветку branch2, разрешив конфликты при этом.

Примечание: конфликт должен произойти обязательно.

7. Выгрузите все изменения во всех ветках в удаленный репозиторий.

8. Проведите еще 3 коммита в ветку branch1.

9. Склонируйте репозиторий еще раз в другую директорию.

10. В новом клоне репозитория сделайте 3 коммита в ветку branch1.

11. Выгрузите все изменения из нового репозитория в удаленный репозиторий

12. Вернитесь в старый клон с репозиторием, выгрузите изменения с опцией –force.

13. Получите все изменения в новом репозитории.

1.6. Вывод

Практическая работа направлена на знакомство с системой контроля версий Git, включая установку и настройку Git, работу с локальными и удаленными репозиториями (GitHub), изучение ветвления и слияния, а также решение конфликтов.

1.7. Контрольные вопросы

1. Какие существуют типы систем контроля версий? Приведите примеры к каждому типу.

2. К какому типу систем контроля версий относится Git?

3. Что такое репозиторий Git?

4. Что такое коммит?

5. Что такое ветка в репозитории Git?

6. Что такое тег в репозитории Git?

7. Что такое слияние двух веток?

8. Что такое конфликт в Git? Как его решить и почему они бывают?

9. Как отменить слияние веток, если произошел конфликт?

10. Для чего нужен .gitignore?

11. Что делает команда git status?

12. Что делает команда git add?
13. Что делает команда git log?
14. Что делает команда git diff?
15. Что делает команда git show?
16. Что делает команда git stash?
17. Как добавить новую директорию в Git?
18. Что сделает команда git branch без какого-либо параметра?
19. Чем отличаются команды git push и git pull?
20. Что означает статус файла untracked в выводе команды git status?
21. Что означает статус файла new в выводе команды git status?
22. Что означает статус файла modified в выводе команды git status?
23. Чем отличается master и origin/master?
24. Как узнать, кто автор строчки в файле, используя систему Git?
25. Как отменить действие команды git add на файл?
26. Как привести измененный файл в начальное состояние (до изменения)?
27. Как сделать ветку с названием my_branch?
28. Как сделать коммит для ветки my_branch?
29. Как удалить локальную ветку my_branch?
30. Как исправить ошибку «fatal: The current branch my_branch has no upstream branch», возникающую при вводе git push?
31. Как удалить все untracked файлы?
32. Что такое GitHub?
33. Что такое форк репозитория?
34. Что такое pull request?

1.8. Требования к отчету

По итогу выполнения практической работы необходимо оформить отчет, включающий:

1. Титульный лист.
2. Оглавление.
3. Цели и задачи работы.
4. Выполнение 1, 2, 3 и 4 частей задания с описанием шагов и скриншотами используемых команд.

Примечание: необходимо удостовериться, что выполнены все пункты самого задания.

5. Ответы на контрольные вопросы (8 любых) с развернутым пояснением.

6. Вывод о проделанной работе.

Отчет о практической работы необходимо загрузить в Систему дистанционного обучения и очно защитить на паре.

1.9. Список источников для изучения

1. Жматов, Д. В. GIT: Создание прочной основы для эффективной разработки : учебное пособие / Д. В. Жматов. — Москва : РТУ МИРЭА, [б. г.]. — Часть 1 — 2024. — 114 с. — ISBN 978-5-7339-2345-1.

2. Atlassian Git Tutorial: Разрешение конфликтов слияния [Электронный ресурс] // Atlassian. – Режим доступа: https://www.atlassian.com/ru/git/tutorials/using-branches/merge-conflicts (дата обращения: 20.04.2025).

3. Документация GitHub [Электронный ресурс] // GitHub, Inc. – Режим доступа: https://docs.github.com/ru (дата обращения: 20.04.2025).

2. СИСТЕМЫ СБОРКИ

Цель практической работы: автоматизация процесса сборки, управление зависимостями и развертывание программных проектов. Это позволяет разработчикам сократить время на рутинные задачи, повысить надежность и гибкость сборочного процесса.

2.1. Теоретическая часть

Gradle — система сборки проектов с открытым исходным кодом, в которой основной упор идет на гибкость и производительность. В настоящее время поддерживаются 2 языка программирования для написания скриптов сборки: groovy и kotlin.

Для работы с Gradle нужна только установленная jdk, для всех проектов хорошим тоном считается иметь gradle-wrapper. Это обертка из небольшого jar-файла (порядка 58 Кб), 1 файла конфигурации и 2 скриптов для запуска — windows и *nix. При первом запуске враппера загружается gradle необходимой версии и после чего начинает свою штатную работу. Данная схема необходима, так как в зависимости от множества причин версия системы сборки у членов команды и билд-сервера может отличаться.

Gradle далеко не первая система сборки для java и Android-приложений, поэтому данная система может использовать хранилища артефактов (библиотек), которые изначально предназначались для Maven.

Вызов враппера в корне проекта:

а) в Windows: gradlew.bat (в PowerShell ./gradlew.bat);

б) в Unix-like: ./gradlew.

Если все сделано правильно, то gradle вам сообщит свою версию и выведет небольшой help по своим командам (рис. 2.1):

```
Welcome to Gradle 6.8.

To run a build, run gradlew <task> ...

To see a list of available tasks, run gradlew tasks

To see a list of command-line options, run gradlew --help

To see more detail about a task, run gradlew help --task <task>

For troubleshooting, visit https://help.gradle.org

BUILD SUCCESSFUL in 2s
```

Рисунок 2.1. Результат вызова враппера

Задачи в gradle выполняются следующим образом: вызов враппера gradle задача1 задача2 задачаN. Выполнение задач будет именно в той последовательности, в которой Вы их вызовете. Также некоторые задачи по умолчанию зависят друг от друга. Например, нет необходимости вызывать задачу compileJava, если Вы собираете проект целиком командой build.

Самые популярные задачи:

- clean — очистка всех сборочных директорий;
- build — собрать приложение целиком;
- javadoc — сгенерировать JavaDoc документацию;
- shadowJar — собрать UberJar (архив со всеми зависимостями, для использования необходимо импортировать плагин shadow);
- run — запуск приложения;
- test — прогон тестов.

Рассмотрим базовый скрипт для сборки при создании проекта в Intellij IDEA (для данного скрипта используется язык Groovy) на рис. 2.2:

```
plugins {
    id 'java'
}

group 'org.text'
version '1.0'

    repositories { ma-
venCentral()
}

dependencies {
    testImplementation 'org.junit.jupiter:junit-jupiter-
api:5.6.0' testRuntimeOnly 'org.junit.jupiter:junit-jupiter-en-
gine'
}
```

Рисунок 2.2. Пример скрипта сборки

В данном случае рассмотрены следующие разделы:

1. plugins — здесь перечисляются плагины, которые будут использоваться при сборке проекта. В сам gradle включены только «Core Plugins». Если для нужд проекта чего-либо не хватает, то можно воспользоваться поиском на https://plugins.gradle.org/, либо реализовать свой плагин, который будет решать поставленную задачу.
2. Пакет и версия нужны для идентификации вашего проекта. Если кто-то публикует приложение или библиотеку в maven-репозиториях, то по пакету и названию проекта можно его найти (указывается в рядом лежащем settings.gradle, например: rootProject.name = 'sample').
3. repositories — здесь перечисляются репозитории, которые будут использоваться для поиска и загрузки артефактов. Самым популярным является mavenCentral(). Если у Вас есть зависимости от других Ваших проектов, которые вы пока нигде не публикуете, то не забудьте указать еще mavenLocal() — в таком случае поиск артефактов будет производиться и по Вашим локально собранным пакетам. Если Вам необходимо указывать зависимости, которые есть, например, только в Вашем рабочем хранилище артефактов, то такие зависимости указываются следующим образом внутри блока repositories:

```
maven {
     url "https://plugins.gradle.org/m2/"
}
```

4. dependencies — зависимости проекта. Скорее всего, когда приложение становится чуть больше HelloWorld, возникает необходимость в импортировании кода от третьих лиц. Наиболее популярные типы:

а) annotationProcessor — обработка аннотаций в процессе компиляции (чаще всего используется для кодогенерации)

б) implementation — зависимости, необходимые для компиляции проекта, которые не являются частью API, предоставляемого проектом. Например, проект использует Hibernate для работы с БД, но API для работы с ним не используется.

в) api — зависимости, необходимые для компиляции проекта, которые являются частью API, предоставляемого проектом. Например, проект использует Guava и предоставляет публичные интерфейсы с классами Guava в их сигнатурах методов.

Также чаще всего в проектах указывают следующие разделы:

1. Для того, чтобы явно указать версию java для работы необходимо использовать:

```
java {
     sourceCompatibility = JavaVersion.toVersion("15")
     targetCompatibility = JavaVersion.toVersion("15")
}
```

2. Для указания кодировки (особенно актуально, у кого комментарии, javadoc или строки в хранилище текста могут быть на русском языке) используется:

```
compileJava {
     options.encoding = 'UTF-8'
}
compileTestJava {
     options.encoding = 'UTF-8'
}
javadoc {
     options.encoding = 'UTF-8'
}
```

2.2. Практическая часть

Для выполнения практической части необходимо клонировать (или форкнуть) git-репозиторий согласно варианту. Перед сборкой проекта нужно проверить настройки: установленный Java Development Kit (jdk) и версию Gradle 6.8. Рекомендовано использование jdk версии 11, поскольку она совестима с версией Gradle 6.8. Стоит учитывать, что при повышении версии jdk, также стоит повысить версию Gradle. Рис. 2.3 отражает совместимость версий:

Gradle Version	Supported Java Versions
Gradle 6.x	8, 9, 10, 11
Gradle 7.0	8, 11, 15
Gradle 7.1–7.3	8, 11, 15
Gradle 7.4–7.6	8, 11, 15, 16
Gradle 7.6.1	8, 11, 15, 16, 17
Gradle 8.0	17–20
Gradle 8.1–8.3	17–20

Рисунок 2.3. Совместимость версий

В файле для автоматической загрузки Gradle gradlewrapper.properties в distributionUrl указана версия 6.8. Gradle Wrapper автоматически скачивает Gradle, если он не установлен локально (рис. 2.4).

```
1  distributionBase=GRADLE_USER_HOME
2  distributionPath=wrapper/dists
3  distributionUrl=https\://services.gradle.org/distributions/gradle-6.8-bin.zip
4  zipStoreBase=GRADLE_USER_HOME
5  zipStorePath=wrapper/dists
```

Рисунок 2.4. Файл gradlewrapper.properties

В случае, если в системе несколько версий Java, а также если версия Java не определяется автоматически, рекомендуется настроить системную переменную JAVA_HOME со значением корректного пути к нужной папке JDK. Например, C:\Program Files\Java\jdk-11.0.17.

2.2.1. Задание на практическую работу

Практическая часть состоит из следующих заданий:

1. Первая попытка сборки проекта через задачу build приведет к набору ошибок. Исправив все ошибки, необходимо продемонстрировать успешную сборку. Также в отчете необходимо показать исправление всех ошибок.

Некоторые из ошибок при сборке:

1. Найти отсутствующую зависимость и указать ее в соответствующем блоке в build.gradle.
2. В некоторых классах поправить имя пакета.
3. Добавить импорт отсутствующего класса (в зависимости от варианта задания).

Примечание: в некоторых вариантах ошибки могут отличаться. Необходимо продемонстрировать решение всех из них.

4. Собрать документацию проекта, найти в ней запросы состояния и сущности по идентификатору.
5. Собрать jar со всеми зависимостями (так называемый UberJar), после чего запустить приложение. По умолчанию, сервер стартует на порту 8080.
6. Запросить состояние запущенного сервера (GET запрос по адресу http://localhost:8080/)
7. Запросить сущность по идентификатору (GET запрос по адресу: http://localhost:8080/сущность/идентификатор). Идентификатором будут 3 последние цифры в серийном номере Вашего студенческого билета.
8. В задаче shadowJar добавить к jar-файлу Вашу фамилию. Также продемонстрировать на рисунке созданный jar-файл с фамилией.
9. Выполнить задачу checkstyleMain. Посмотреть сгенерированный отчет. Устранить ошибки оформления кода. Отобразить сформированную таблицу с количеством ошибок и таблицу с отсутствием ошибок.

Примечание: некоторые из ошибок могут остаться даже при успешной сборке проекта на 1 этапе. Сгенерированный отчет должен показывать, что все ошибки исправлены.

2.2.2. Варианты для выполнения работы

1. Репозиторий: https://github.com/trppmosit/trpp-second-1

Сущность: ru.mirea.entity.Employee

2. Репозиторий: https://github.com/trppmosit/trpp-second-2

Сущность: ru.mirea.entity.Student

3. Репозиторий: https://github.com/trppmosit/trpp-second-3

Сущность: ru.mirea.entity.Client

4. Репозиторий: https://github.com/trppmosit/trpp-second-4

Сущность: ru.mirea.entity.Organization

5. Репозиторий: https://github.com/trppmosit/trpp-second-5

Сущность: ru.mirea.entity.Medicine

6. Репозиторий: https://github.com/trppmosit/trpp-second-6

Сущность: ru.mirea.entity.Message

7. Репозиторий: https://github.com/trppmosit/trpp-second-7

Сущность: ru.mirea.entity.Movie

8. Репозиторий: https://github.com/trppmosit/trpp-second-8

Сущность: ru.mirea.entity.Book

9. Репозиторий: https://github.com/trppmosit/trpp-second-9

Сущность: ru.mirea.entity.Nomenclature

10. Репозиторий: https://github.com/trppmosit/trpp-second-10

Сущность: ru.mirea.entity.Game

11. Репозиторий: https://github.com/trppmosit/trpp-second-11

Сущность: ru.mirea.entity.Device

12. Репозиторий: https://github.com/trppmosit/trpp-second-12

Сущность: ru.mirea.entity.Animal

13. Репозиторий: https://github.com/trppmosit/trpp-second-13

Сущность: ru.mirea.entity.Car

14. Репозиторий: https://github.com/trppmosit/trpp-second-14

Сущность: ru.mirea.entity.History

15. Репозиторий: https://github.com/trppmosit/trpp-second-15

Сущность: ru.mirea.entity.Receipt

2.3. Вывод

Работа с Gradle направлена на оптимизацию и ускорение процесса разработки программного обеспечения. Благодаря автоматизации рутинных задач, удобному управлению зависимостями и высокой производительности, Gradle является мощным инструментом для разработчиков.

2.4. Контрольные вопросы

1. Чем компиляция отличается от сборки?
2. Что такое система сборки?
3. Что такое репозиторий?
4. Как указать зависимости проекта?

5. Что такое gradle?
6. Что такое maven?
7. Что такое mavencentral?
8. Что делает задача clean?
9. Что делает задача build?
10. Что делает задача compileJava?
11. Что делает задача run?
12. Что делает задача test?
13. Что такое javadoc?
14. Что такое checkstyle?
15. Что такое UberJar? При помощи какой задачи его собрать?
16. Что такое micronaut?
17. Что такое lombok?
18. Что такое postman?
19. Что такое аннотация в Java?

2.5. Требования к отчету

По итогу выполнения практической работы необходимо оформить отчет, включающий:

1. Титульный лист.
2. Оглавление.
3. Цели и задачи работы.
4. Выполнение всех пунктов задания.
5. Письменные развернутые ответы на все контрольные вопросы.
6. Вывод о проделанной работе.

Отчет о практической работы необходимо загрузить в Систему дистанционного обучения и очно защитить на паре.

2.6. Список источников для изучения

1. Документация Gradle [Электронный ресурс] // Gradle. – Режим доступа: https://docs.gradle.org/current/userguide/userguide.html (дата обращения: 05.04.2025).
2. Статья по управлению зависимостями проекта [Электронный ресурс] // Gradle. – Режим доступа: https://docs.gradle.org/current/userguide/declaring_dependencies.html (дата обращения: 05.04.2025).

3. MVN Repository [Электронный ресурс] // Maven. – Режим доступа: https://mvnrepository.com/ (дата обращения: 11.03.2025).

4. Документация по gradle-плагину shadow [Электронный ресурс] // GradleUp. – Режим доступа: https://gradleup.com/shadow/getting-started/#__tabbed_1_1 (дата обращения: 01.04.2025).

5. Документация по фреймворку Micronaut Framework Documentation [Электронный ресурс] // Micronaut. – Режим доступа: https://micronaut.io/documentation.html (дата обращения: 11.03.2025).

6. Postman API Platform для тестирования сервера [Электронный ресурс] // Postman Inc. – Режим доступа: https://www.postman.com/ (дата обращения: 24.04.2025).

7. Lombok [Электронный ресурс] // Project Lombok. – Режим доступа: https://projectlombok.org/ (дата обращения: 05.04.2025).

3. DOCKER

Цель практической работы: изучение основных возможностей Docker, включая работу с образами, контейнерами, портами, томами, переменными окружения и создание Dockerfile, приобретение навыков сборки и запуска контейнеров, а также выполнение индивидуального задания с учетом специфики варианта.

3.1. Теоретическая часть

Docker — это платформа, которая позволяет упаковать в контейнер приложение со всем окружением и зависимостями, а затем доставить и запустить его в целевой системе.

При работе с Docker необходимо знать следующие ключевые элементы:

1. Docker Host — операционная система, на которой развернут и функционирует Docker.
2. Docker Daemon (демон) — фоновый сервис, отвечающий за управление объектами Docker: сетями, хранилищами, образами и контейнерами.
3. Docker Client — интерфейс командной строки (CLI), через который пользователи отправляют команды демону для создания и управления контейнерами.
4. Docker Image (образ) — неизменяемый шаблон, используемый для развертывания контейнеров.
5. Docker Container (контейнер) — запущенный экземпляр приложения, созданный на основе образа.
6. Dockerfile — текстовый файл с инструкциями для автоматической сборки образов.
7. Docker Compose — инструмент для оркестрации многоконтейнерных приложений, упрощающий их настройку и запуск.
8. Docker Desktop — графический интерфейс (GUI), распространяемый под лицензией GPL. Доступен для Windows, macOS и Linux, предоставляет удобный способ управления Docker и поддержку локального Kubernetes.

Все команды в нижеперечисленных разделах теоретической части необходимо повторить на своей локальной машине.

Важное примечание: в данном методическом указании к практической работе все представленные команды записаны строго в одну строку — не переносите их в командной строке на несколько строк. Экранирование (\) не

используется. Если хотите написать одну команду в несколько строк используйте обратный слеш (\).

3.1.1. Образы

Для просмотра имеющихся образов используйте команду:

```
docker images
```

Загрузите образ можно с использованием команд:

```
docker pull ubuntu //загрузка последней версии образа ubuntu
docker pull ubuntu:24.04 // загрузка конкретной версии 24.04
```

Посмотрите на имеющиеся образы еще раз через docker images — должны появиться новые загруженные образы.

Посмотрите список контейнеров, выполнив команду:

```
docker ps -a
```

3.1.2. Изоляция

Посмотрите информацию о хостовой системе, выполнив команду:

```
hostname
```

Выполните эту команду еще один раз.

Вопрос: одинаковый ли результат получился при разных запусках? Необходим развернутый ответ в отчете.

Попробуйте выполнить то же самое в контейнерах. Выполните два раза команду:

```
docker run ubuntu hostname
```

Вопрос: одинаковый ли результат получился при разных запусках? Необходим развернутый ответ в отчете.

В случае запуска команды в контейнерах ответ будет немного отличаться, будет разный hostname. Так происходит, потому что из одного образа ubuntu были запущены два изолированных контейнера, поэтому у них и был разный hostname.

Заново выполните команду docker ps -a — там должны появиться запущенные ранее контейнеры.

Запуск контейнеров производится командой:

```
docker run --флаги --докера имя_контейнера команда для запуска -и
--флаги --запуска --программы
```

Запустите bash в контейнере:

```
docker run ubuntu bash
```

После выполнения команды ничего не должно произойти. Интерактивные оболочки выйдут после выполнения любых скриптовых команд, если только они не будут запущены в интерактивном терминале — поэтому для того, чтобы этот пример не завершился, нужно добавить флаги -i и -t, или сгруппированно -it:

```
docker run -it ubuntu bash
```

Выполняя запуск контейнера, указывая образ ubuntu, неявно указывался образ ubuntu:latest. Следовательно, следующие команды равнозначны:

```
docker run ubuntu hostname
docker run ubuntu:latest hostname
```

Если бы необходимо запустить ubuntu:24.04, то команда будет следующая:

```
docker run ubuntu:24.04 hostname
```

3.1.3. Работа с портами

Для начала, загрузите образ python с использованием команды:

```
docker pull python
```

В качестве примера, запустите встроенный в Python модуль веб-сервера из корня контейнера, чтобы отобразить содержание контейнера:

```
docker run -it python python -m http.server
```

При запуске пишется, что сервер доступен по адресу http://0.0.0.0:8000/. Однако, если открыть этот адрес, то ничего не будет видно, потому что порты не проброшены. Завершите работу веб-сервера, нажав комбинацию клавиш Ctrl+C.

Для проброса портов используется флаг -p hostPort:containerPort.

Необходимо добавить его, пробросив порт 8000:

```
docker run -it -p8000:8000 python python -m http.server
```

Теперь по адресу http://0.0.0.0:8000/ открывается содержимое корневой директории в контейнере (если не открывается на Windows, то вместо 0.0.0.0 нужно указать localhost).

Для того, чтобы доступный в контейнере на порту 8000 веб-сайт в хостовой системе открывался на порту 8888, необходимо указать флаг -p8888:8000:

```
docker run -it -p8888:8000 python python -m http.server
```

Завершите работу веб-сервера, нажав комбинацию клавиш Ctrl+C.

3.1.4. Именованные контейнеры, остановка и удаление

Запустите контейнер с помощью команды:

```
docker run -it -p8000:8000 python python -m http.server
```

Нажмите Ctrl+C выполнение завершится. Для того, чтобы запустить контейнер в фоне, нужно добавить флаг-d/--detach. Для определения имени контейнера добавляется флаг --name. Команда с флагами выглядит следующим образом:

```
docker run -p8000:8000 --name pyserver -d python python -m
http.server
```

Убедитесь, что контейнер все еще запущен с использованием одной из команд (вывод команды не должен быть пустым):

```
docker ps | grep pyserver //команда для Unix/Linux, Git Bash Win-
dows
docker ps | findstr "pyserver" //команда для Windows CMD
```

Для просмотра логов контейнера, воспользуйтесь командой:

```
docker logs pyserver
```

Для того, чтобы остановить выполнение контейнера, выполните команду:

```
docker stop pyserver
```

Запустите контейнер еще раз:

```
docker run -it -p8000:8000 --name pyserver -d python python -m
http.server
```

Должна возникнуть ошибка, что контейнер с таким именем уже существует. Необходимо удалить контейнер:

```
docker rm pyserver
```

Вместо выполнения двух отдельных команд stop и rm для остановки и удаления контейнера можно воспользоваться командой:

```
docker rm -f pyserver
```

После удаления контейнер с таким именем можно будет создать заново.

Для того, чтобы контейнер удалялся после завершения работы, нужно указать флаг --rm при его запуске:

```
docker run --rm -p8000:8000 --name pyserver -d python python -m
http.server
```

3.1.5. Постоянное хранение данных

Запустите контейнер, в котором веб-сервер будет отдавать содержимое директории /mnt:

```
docker run -p8000:8000 --name pyserver --rm -d python python -m
http.server -d /mnt
```

Флаг -d /mnt указывает модулю http.server, какая директория будет корневой для отображения.

Вопрос: что значат остальные флаги запуска? Где здесь команда, которая выполнится в контейнере? Необходим развернутый ответ в отчете.

Для того, чтобы попасть в уже запущенный контейнер, существует команда:

```
docker exec -it pyserver bash
```

Команда позволит попасть в оболочку bash в контейнере. Попав в контейнер, выполните команду:

```
cd mnt && echo "hello world" > hi.txt
```

Затем выйдите из контейнера, введя команду exit или нажав комбинацию клавиш Ctrl+D.

Если открыть http://0.0.0.0:8000/, там будет доступен файл hi.txt.

Остановим контейнер:

```
docker stop pyserver
```

И затем снова его запустим:

```
docker run -p8000:8000 --name pyserver --rm -d python python -m
http.server -d /mnt
```

Как мы видим, файл hi.txt пропал, так как мы запустили другой контейнер, а старый был удален после завершения работы (флаг --rm).

Остановим контейнер через docker stop pyserver.

Для того, чтобы не терялись какие-то данные, существует механизм монтирования.

3.1.6. Тома

Первый способ — это создать отдельный том с помощью ключа -v myVolume:/mnt, где myvolume — название тома, /mnt — директория в контейнере, где будут доступны данные.

Попробуйте снова создать контейнер, но уже с примонтированным томом:

```
docker run -p8000:8000 --rm --name pyserver -d -v myVolume:/mnt
python python -m http.server -d /mnt
```

Затем нужно выполнить docker exec -it pyserver bash и внутри контейнера прописать cd mnt && echo "hello world" > hi.txt. Даже после удаления контейнера данные в этом томе будут сохранены.

Чтобы узнать, где хранятся данные, выполните команду:

```
docker inspect -f "{{json .Mounts}}" pyserver
```

В поле Source будет храниться путь до тома на хостовой машине.

Для управления томами существует команда (ознакомиться с ней можно самостоятельно):

```
docker volume
```

3.1.7. Монтирование директорий и файлов

Для начала необходимо остановить контейнер, созданный на предыдущем шаге с использованием docker stop pyserver.

Иногда требуется пробросить в контейнер конфигурационный файл или отдельную директорию. Для этого используется монтирование директорий и файлов.

Создадим директорию и файл, которые будем монтировать:

```
mkdir myfiles
cd myfiles
touch host.txt
```

Запустите контейнер с помощью команды ниже (корректная команда с pwd для Unix-подобных операционных систем). Команда pwd выведет текущую директорию, например: /home/user/dome-directory. В итоге получиися абсолютный путь до файла: /home/user/dome-directory/myfiles.

```
docker run -p8000:8000 --rm --name pyserver -d -v
$(pwd)/myfiles:/mnt python python -m http.server -d /mnt
```

Если работа выполняется на Windows CMD, то команда запуска будет следующая (вместо pwd добавляете полный путь до директории):

```
docker run -p8000:8000 --rm --name pyserver -d -v
"C:\путь\до\директории:/mnt" python python -m http.server -d /mnt
```

Затем зайдите в контейнер с использованием docker exec -it pyserver bash, перейдите в директорию /mnt командой cd /mnt. Если вывести список файлов командой ls, то там будет файл host.txt, примонтированный вместе с директорией myfiles.

Создайте файл с использованием echo "hello world" > hi.txt, а затем выйдите из контейнера: exit. Теперь на хостовой машине в директории myfiles/ появится файл hi.txt. Проверить можно командой ls myfiles.

Остановите контейнер с использованием команды docker stop pyserver.

Для того, чтобы примонтировать один файл, нужно указать ключ -v и путь до файла, например:

```
docker run -p8000:8000 --rm --name pyserver -d -v
$(pwd)/myfiles/host.txt:/mnt/new-name-of-host.txt python python -m
http.server -d /mnt
```

При этом в данной команде файлу в контейнере присвоится другое имя: new-name-of-host.txt.

Примечание: если на Windows возникают ошибки при монтировании, убедитесь, что можно использовать bash, а не cmd.exe.

3.1.8. Переменные окружения

Для передачи переменных окружения внутрь контейнера используется ключ -e. Например, чтобы передать в контейнер переменную окружения MIREA со значением «ONE LOVE», нужно добавить ключ -e MIREA="ONE LOVE".

Проверьте команду, выведя все переменные окружения, определенные в контейнере с помощью утилиты env:

```
docker run -it --rm -e MIREA="ONE LOVE" ubuntu env
```

Среди списка переменных будет и MIREA.

3.1.9. Dockerfile

Соберите образ, в который будут установлены дополнительные пакеты, примонтируйте директорию и установите команду запуска. Для этого создается файл Dockerfile (без расширения).

```
FROM ubuntu:22.04
RUN apt update \
&& apt install -y python3 fortune \
&& cd /usr/bin \
&& ln -s python3 python
RUN /usr/games/fortune > /mnt/greeting-while-building.txt
ADD ./data /mnt/data
EXPOSE 80
CMD ["python", "-m", "http.server", "-d", "/mnt/", "80"]
```

Не забудьте создать директорию data, которая должна лежать в одной директории с Dockerfile.

Dockerfile должен выглядеть следующим образом (рис. 3.1):

```
C: > Users > Anna Ptrv > Desktop > TRPP > docker > новая > Dockerfile > ...
1  FROM ubuntu:22.04
2  RUN apt update \
3  && apt install -y python3 fortune \
4  && cd /usr/bin \
5  && ln -s python3 python
6  RUN /usr/games/fortune > /mnt/greeting-while-building.txt
7  ADD ./data /mnt/data
8  EXPOSE 80
9  CMD ["python", "-m", "http.server", "-d", "/mnt/", "80"]
```

Рисунок 3.1. Dockerfile

В строке (1) указывается базовый образ, на основе которого будет строиться новый образ.

В строках (2-5) указана команда, которая выполнится в процессе сборки. На самом деле, там выполняются несколько команд, соединенных && для того, чтобы создавать меньше слоев в образе.

В строках (6) тоже указана команда, которая сгенерирует случайную цитату и перенаправит вывод в файл /mnt/greeting-while-building.txt. Файл будет сгенерирован во время сборки образа.

В строке (7) копируется все содержимое директории ./data хостовой машины в директорию /mnt, которая будет доступна в контейнере.

В строке (8) указывается, какой порт у контейнера будет открыт.

В строке (9) указывается команда, которая будет выполнена при запуске, где 80 — порт, который будет слушать веб-сервер.

Примечание: будьте внимательны при копировании, могут скопироваться неправильные минусы и лишние пробелы.

Далее соберите образ с тегом mycoolimage с помощью команды:

```
docker build -t mycoolimage .
```

Точка в конце указывает на текущую директорию, где лежит Dockerfile.

Запуск контейнера производится командой (порт 8099 — порт на хостовой машине):

```
docker run --rm -it -p8099:80 mycoolimage
```

Переход по указанному порту должен быть успешным.

3.2. Практическая часть

В практической работе необходимо выполнить все шаги из разделов 4.1.1–4.1.9.

В отчет должны быть включены ответы на вопросы, выделенные курсивом, результаты выполнения команд из разделов 4.1.1–4.1.9, а также выполненное индивидуальное задание (описано ниже): листинг Dockerfile, а также команды сборки и запуска контейнера.

Для установки Docker в Windows необходимо установить подсистему Windows для Linux, воспользовавшись инструкцией по ссылке: https://learn.microsoft.com/ru-ru/windows/wsl/install. А также нужно установить Docker Desktop по ссылке: https://learn.microsoft.com/ru-ru/windows/wsl/tutorials/wsl-containers#install-docker-desktop.

Если нет возможности установить Docker, то можно воспользоваться сервисом Play With Docker по ссылке: https://labs.play-with-docker.com/:

1. Нажать на кнопку Login (рис. 3.2):

Рисунок 3.2. Кнопка Login

Входим в аккаунт или создаем новый, если его не было раньше (рис. 3.3):

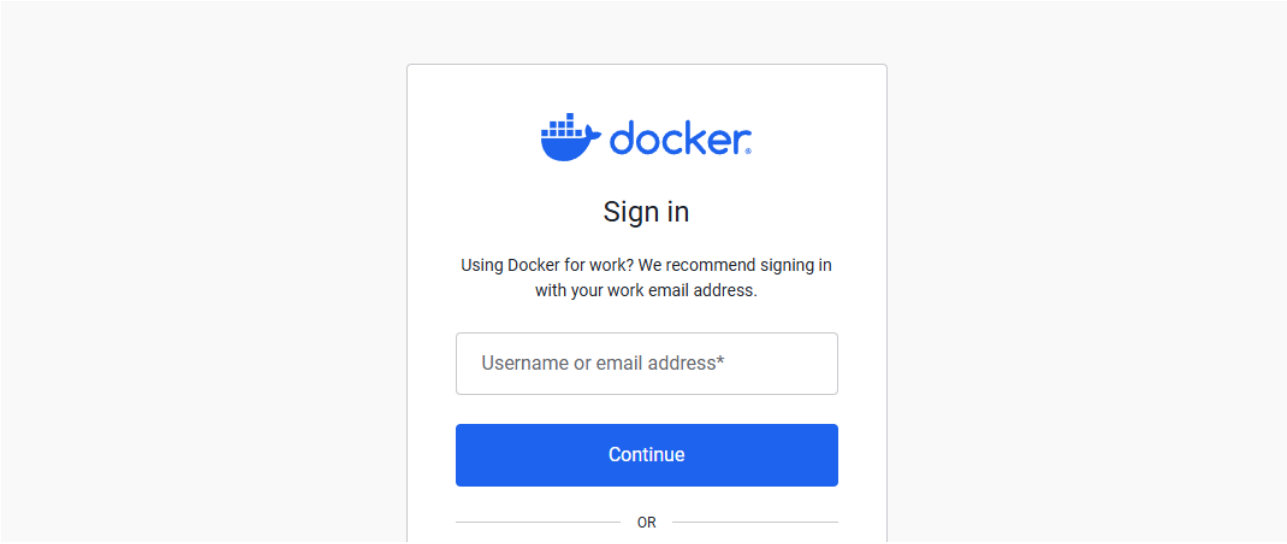

Рисунок 3.3. Вход в аккаунт Docker

Далее нажимаем на кнопку «Старт» (рис. 3.4):

Рисунок 3.4. Кнопка Start

Создаем новый инстанс (будет доступен с момента создания в течение 4-х часов), клинкув по «Add new instance» (рис. 3.5):

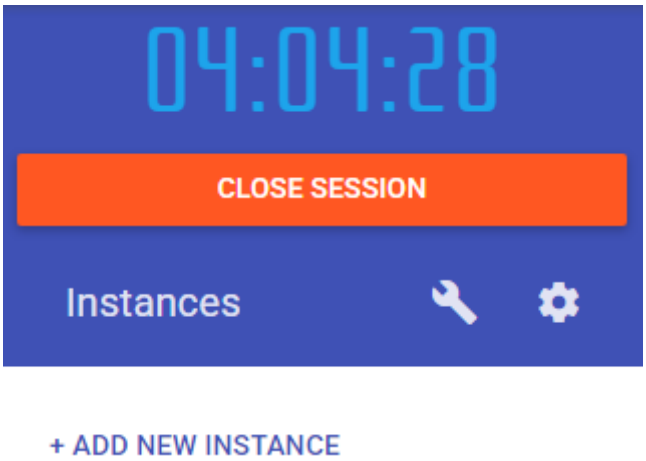

Рисунок 3.5. Add new instance

Работу можно начать в следующей командной строке (рис. 3.6):

```
[node1] (local) root@192.168.0.13 ~
$ docker images
REPOSITORY    TAG        IMAGE ID    CREATED    SIZE
[node1] (local) root@192.168.0.13 ~
$
```

Рисунок 3.6. Начало работы

Индивидуальное задание

Для практической работы необходимо написать Dockerfile, собрать образ, запустить контейнер.

Для монтирования создайте директорию data и в ней файл student.txt, содержащий ФИО, название группы и номер варианта.

Примечание: директория data должна лежать в одной директории с Dockerfile.

Для установки пакетов использовать команду apt install -y название-пакета. В качестве примера можно использовать Dockerfile из раздела 4.1.9.

Работа с Dockerfile меняется в зависимости от варианта:

1. Четные варианты:

а) необходимо использовать базовый образ ubuntu:latest;

б) примонтировать файл data/student.txt как /mnt/files/student.txt в контейнере.

2. Нечетные варианты:

а) необходимо использовать базовый образ ubuntu:24.04;

б) примонтировать директорию data в директорию /mnt/files/ в контейнере.

После написания Dockerfile необходимо собрать образ и запустить веб-сервер, отображающий содержимое /mnt/files.

В хостовой системе сервер должен открываться на порту (8800 + номер варианта). Например, для 22-го варианта это порт 8822.

Четность и нечетность варианта определяется вариантом, который указывается для порта.

Установить пакет нужно согласно следующему варианту:

1. cowsay,
2. figlet,
3. zip,
4. imagemagick,
5. git,
6. patch,
7. php-cli,
8. postgresql-client,
9. mysql-client,
10. jq,
11. gpg,
12. wget,
13. nginx,
14. nano,
15. emacs-nox.

3.3. Вывод

Практическая работа позволяет закрепить навыки работы с Docker, включая создание, настройку и управление контейнерами, а также использование Dockerfile для автоматизации сборки образов. Полученные знания могут быть применены для развертывания и управления приложениями в изолированных средах.

3.4. Контрольные вопросы

1. Что такое Docker?
2. Чем контейнер Docker отличается от виртуальной машины?
3. Что такое образ Docker?

4. Что такое Dockerfile и для чего он нужен?
5. Какие основные инструкции Dockerfile Вы знаете?
6. Что такое Docker compose?
7. Как запустить контейнер в фоновом режиме?
8. Как пробросить порт из контейнера на хост-систему?
9. Как можно просмотреть список всех работающих контейнеров?
10. Что делает команда docker exec -it <контейнер> bash?
11. Почему данные, созданные в контейнере, могут исчезнуть после его остановки?
12. Как остановить и сразу удалить контейнер одной командой?
13. Что такое монтирование тома в Docker и зачем оно используется?
14. В чем разница между томами (volumes) и монтированием директорий?
15. Что такое переменные окружения в Docker?
16. Для чего используется флаг --rm при запуске контейнера?
17. Для чего используется инструкция EXPOSE в Dockerfile?
18. Как уменьшить количество слоев в образе при выполнении нескольких команд RUN?

3.5. Требования к отчету

По итогу выполнения практической работы необходимо оформить отчет, включающий:

1. Титульный лист.
2. Оглавление.
3. Цели и задачи работы.
4. Выполнение всех шагов из разделов 4.1.1–4.1.9.
5. Выполнение индивидуального задания (листинг Dockerfile, а также выполненные команды сборки и запуска контейнера).
6. Письменные развернутые ответы на все контрольные вопросы.
7. Вывод о проделанной работе.

Отчет о практической работы необходимо загрузить в Систему дистанционного обучения и очно защитить на паре.

ЗАКЛЮЧЕНИЕ

Выполнение практических работ позволяет студентам освоить ключевые инструменты и технологии, используемые в современной разработке программного обеспечения.

Данный практикум закрепляет следующие навыки студентов:

Уметь устанавливать и настраивать системы контроля версий Git, а также выполняют базовые и продвинутые операции: инициализация репозитория, создание и клонирование локальных и удалённых репозиториев, работа с ветвлениями, слияние изменений, разрешение конфликтов использование тегов и инструментов журналирования (логов). Также рассматриваются лучшие практики командной работы с использованием платформ GitHub или GitLab.

Знать основы автоматизации сборки программных проектов с помощью системы Gradle. В рамках практикума выполняются задания по созданию файлов конфигурации, подключению зависимостей, управлению модулями, созданию задач, а также генерации и публикации документации. Практическое применение Gradle помогает понять архитектуру проектов и повысить эффективность процесса сборки и сопровождения программного продукта.

Владеть базовыми концепциями контейнеризации, включая различия между виртуализацией и контейнерами, преимущества использования контейнеров в разработке и эксплуатации ПО. Студенты учатся создавать Dockerfile, собирать образы, запускать и управлять контейнерами, а также взаимодействовать с Docker. Отдельное внимание уделяется построению среды разработки и тестирования на основе контейнеров, что является важным навыком в DevOps-практике.

СПИСОК ИСТОЧНИКОВ ЛИТЕРАТУРЫ

1. Жматов Д. В. GIT: Создание прочной основы для эффективной разработки. Часть 1 : Учебное пособие / Жматов Д. В.. - Москва : РТУ МИРЭА, 2024. - 114 с. — ISBN 978-5-7339-2345-1.
2. Чакон С., Штрауб Б. Pro Git [Электронный ресурс]. — 2-е изд. — Режим доступа: http://www.w1hkj.com/docs/pro_git.pdf, свободный.
3. Малахов, С. В. Принципы работы операционной системы Linux. Bash-скрипты : учебное пособие / С. В. Малахов, Д. О. Якупов. — Самара : ПГУТИ, 2024. — 134 с. — ISBN 978-5-907336-50-6.
4. Gradle in Action [Электронный ресурс] / B. Muschko. – Manning Publications. Режим доступа: https://gradle.org/books/manning-publications-gradle-in-action.pdf, свободный.
5. Моуэт, Э. Использование Docker / Э. Моуэт ; научный редактор А. А. Маркелов ; перевод с английского А. В. Снастина. — Москва : ДМК Пресс, 2017. — 354 с. — ISBN 978-5-97060-426-7.
6. Сейерс, Э. Х. Docker на практике / Э. Х. Сейерс, А. Милл ; перевод с английского Д. А. Беликов. — Москва : ДМК Пресс, 2020. — 516 с. — ISBN 978-5-97060-772-5.
7. Документация Docker [Электронный ресурс] // Docker. – Режим доступа: https://docs.docker.com/ (дата обращения: 15.04.2025).
8. Основные инструкции Docker [Электронный ресурс] // Tproger. – Режим доступа: https://tproger.ru/translations/docker-instuction (дата обращения: 15.04.2025).

ИНФОРМАЦИЯ ОБ АВТОРАХ

Жматов Дмитрий Владимирович, кандидат технических наук, доцент, доцент кафедры математического обеспечения и стандартизации информационных технологий, Институт информационных технологий «МИРЭА – Российский технологический университет» (119454, Россия, Москва, пр-т Вернадского, д. 78).

Петрова Анна Александровна, ассистент кафедры математического обеспечения и стандартизации информационных технологий, Институт информационных технологий «МИРЭА – Российский технологический университет» (119454, Россия, Москва, пр-т Вернадского, д. 78).